

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	10000K	Yes
6	No	NULL	60K	No
7	Yes	Divorced	220K	NULL
8	No	Single	85K	Yes
9	No	Married	90K	No
10	No	Single	90K	No

Mean: 1090K

Trimmed mean (remove min, max): 105K

Median: $(90+100)/2 = \underline{95K}$

Note by Nam



- Classification, Regression,.....: là bài toán Supervised

▼ Lecture 1 - Introduction:

▼ A. What is data mining:



Definition: data mining must **efficient** (data must large).

→ Data must be a **collective intelligence**

⇒ Focus on harness the **unexpected and useful** data (if data are "explored" → it's less value).

- "Data mining is **the discovery of models** for data"
 - Models that **Explain** the data (e.g., a single function)
 - **Predict** the future data instances.

- **Summarize** the data
- **Extract** the most prominent **features** of the data.



- **Analytics ≠ Mining**

- Analysis: a process of examining data to **extract** meaningful **patterns, trends, and knowledge** for decision-making.
- Mining: **discovering hidden patterns, correlations, and useful insights** from large datasets, often using algorithms and machine learning → finding insight → useful information.

- DIKW Pyramid:

- data (raw data)
 - information (processed data)
 - knowledge (insight & pattern)
 - wisdom (Strategic Decision-Making).
-
- Knowledge = insight → human analyze to detect.
E.G: "Customers buying laptops often purchase accessories like mice or headphones." (Market Basket Analysis).
 - Wisdom: create the strategic
E.G: "We should bundle laptops with accessories and offer discounts to boost sales." (Business strategy.)



▼ B. Type of data

- **Transaction Data:** data form invoice, order, bank transaction, phone no, message,....
 - Structured Data
 - Example: phone no analysis the relationship
 - phone in working time ⇒ working relationship
 - phone call in many times during the day ⇒ love, family,... relationship
- **Document Data:** data from email, contracts, articles, research paper,...
 - Unstructured or semi-structured.
 - No SQL → non - structured (JSON, XML, PDFs).
 - TEXT is non - structured data.
- **Network data:** same as graph that has the connection (E.G: Mutual Friends)
- **Medical - Environment - Behavior data - Genomic sequence (DNA),...**

▼ C. What you can do with data 🧐



- Data is **complex, data characteristic:**
 - **Spatial (không gian):** Data with **geographical or spatial**, representing locations, shapes, or distances.
 - GPS. location
 - **Temporal (thời gian): changes over time**
 - Stock prices, weather records, sensor logs.
 - ⇒ use for Time series analysis, trend forecasting, anomaly detection.
 - **Interconnected:** data can be collected multiple - source from phone.
 - phone can collect message, audio, location,... at the same time.

CASE STUDY: Germany 2014 world cup use AI to make decision.

CASE STUDY: Supermarket product placement:

Given the transaction data:

<i>TID</i>	<i>Items</i>
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Product placement

Catalog creation

- Analyze which products should be imported
 - → **Information:** "Milk" need to be imported (base on transaction above).
 - ⇒ **Insight:** if customer buy diaper then milk.
Install nearby stall, let customer buy bread and milk easier.

SOME DATA MINING TECHNIQUE:

- Frequent item sets and Association Rules extraction
 - "Dependency Rule" & "Classic Rule"
 - E.G: If customer buy milk and diaper - then buy beer
 - ⇒ Explain case: newborn → mom take a rest at home → assign for dad to buy stuff
→ supermarket set beer stall near milk.

<i>TID</i>	<i>Items</i>
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Itemsets Discovered:

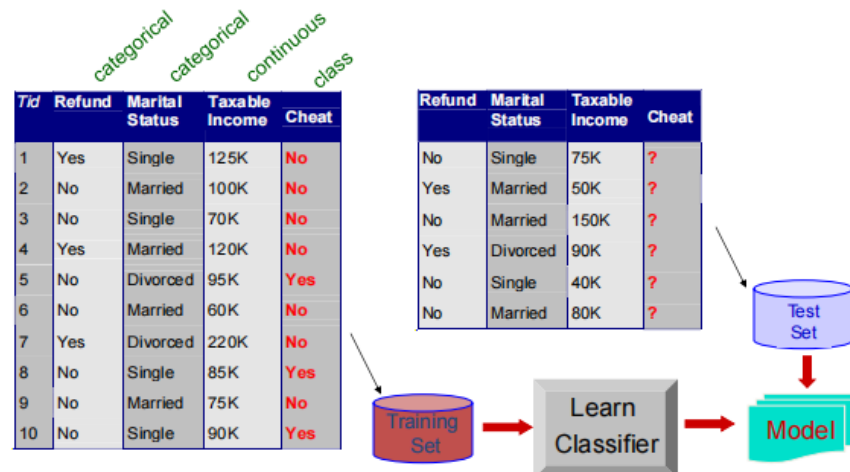
{Milk,Coke}
{Diaper, Milk}

Rules Discovered:

{Milk} --> {Coke}
{Diaper, Milk} --> {Beer}

- **Cluster:**

- The same features combine together.
- Use "Euclidean Distance" to calculate the distance (small distance → similar)
- **Classification:** the join of (Data mining & ML) → Train data → find model → expect to predict the unseen data correctly.



▼ Lecture 2 - What is data:

▼ A. What is data?

- **Data:** Collection of data objects and their attributes.
 - Dataset = A set of records, Data objects
- **Attributes = dimensionality**, fields, variable, characteristic, or features.
- **Object** = record, entity, point, case, sample, or instance
- **Sparsity:** some field that contain info (optional column) → Some field that can "contain nothing"

Attributes

Objects

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Size (n): Number of objects

Dimensionality (d): Number of attributes

Sparsity: Number of populated
object-attribute pairs



- Another call data set:

- **Example: Transaction data**

- Also called **market-basket data**

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

- **Example: Document data**

- Also called **bag-of-words** representation

Doc Id	Words
1	the, dog, followed, the, cat
2	the, cat, chased, the, cat
3	the, man, walked, the, dog

▼ B. Data Attributes

- **Boolean** (not a number attribute) → Special type (depend on context use numeric or categorical).
E.G: Some language convert to 0 and 1; other is true and false.

- Numeric = **vectors**
E.G: structured data → each row = vectors
⇒ distance calculations (e.g., Euclidean distance), **clustering**, and other vector-based operations.
- Age is generally considered a **continuous variable**.

Aspect	Numeric Data = points, vectors	Categorical Data (phân loại): ENUM
Definition	+ Represents measurable quantities (numbers). + Each object is a point in a multidimensional space	+ Represents categories or labels. + Each object is a vector of categorical values
Operations	Arithmetic operations (e.g., addition, subtraction).	Grouping, classification, and frequency counting.
Example	Height, weight, temperature. E.G: 170, 65, 36.5	Eye color, rankings (e.g, good, fair, bad), height in {tall, medium, short}
Subset	Discrete, Continuous , Bool	Nominal, Ordinal

Aspect	Discrete Data	Continuous Data
Definition	Countable, finite values.	Infinite values within a range.
Nature	Integer, no decimals.	Can have decimals or fractions.
Example	Number of Students (1, 2, 3...)	Temperature (25.5°C, 30.8°C...).

Aspect	Nominal Data	Ordinal Data
Definition	without any order.	Categories with a meaningful order.
Ranking	No ranking or hierarchy.	Order.
Example	Eye color (Blue, Green, Brown). ID number, Zip code Phone no	Education level (Primary, Secondary, Tertiary).

1. Numeric Relational Data

- **Definition:** Data where all attributes are **numerical** (either discrete or continuous).
- **Characteristics:** Can be represented in a **multi-dimensional space** for mathematical operations.
- **Example:**

ID	Age	Salary	Height
1	25	3000	170
2	30	5000	180

- Each row (instance) is a **vector** in numerical space.
-

2. Categorical Relational Data

- **Definition:** Data where all attributes are **categorical** (nominal or ordinal).
- **Characteristics:** Categorical values represent groups or rankings, but may not have mathematical meaning.
- **Example:**

ID	Color	Gender	Education Level
1	Red	Male	Bachelor
2	Blue	Female	Master

- Uses **encoding methods** (e.g., one-hot encoding, label encoding) for machine learning.

3. Mixed Relational Data

- **Definition:** Data that contains both **numerical and categorical** attributes.
- **Characteristics:** Requires **hybrid processing techniques** (e.g., encoding categorical values before distance-based operations).
- **Example:**

ID	Age	Gender	Salary	Department
1	25	Male	3000	HR
2	30	Female	5000	IT

- "Age" and "Salary" are **numeric**, while "Gender" and "Department" are **categorical**.
- Categorical attributes are **converted** into numerical form for processing.

Mixed Relational Data

- Data that consists of a collection of records, each of which consists of a **fixed set** of both **numeric** and **categorical** attributes

ID Number	Zip Code	Age	Marital Status	Income	Income Bracket	Refund
1129842	45221	55	Single	250000	High	0
2342345	45223	25	Married	30000	Low	1
1234542	45221	45	Divorced	200000	High	0
1243535	45224	43	Single	150000	Medium	0

Boolean attributes can be thought as both numeric and categorical
When appearing together with other attributes they make more sense as **categorical**
They are often represented as numeric though

▼ C. "Vector" in dataset & How to store data



- Database not for harness - but use only for extract.
 - **Database**: optimize for retrieve and collect data
 - **Mining (in context database)**: process at the same time (order) → exploit data all from top to bottom
→ some time mining use SELECT

- TSV/CSV - Excel sheets, relational tables.
- JSON, XML (standard, flexible)
 - There exist parsers for reading such data.
- Flat file with **triplet** (record id, attribute, attribute value)
 - → solve: some data field is "null" (Ví dụ: hình ở dưới)

Comma Separated File

```
id, Name, Surname, Age, Zip  
1, John, Smith, 25, 10021  
2, Mary, Jones, 50, 96107  
3, Joe, Doe, 80, 80235
```

- Can be processed with simple parsers, or loaded to excel or a database

Triple-store

```
1, Name, John  
1, Surname, Smith  
1, Age, 25  
1, Zip, 10021  
2, Name, Mary  
2, Surname, Jones  
2, Age, 50  
2, Zip, 96107  
3, Name, Joe  
3, Surname, Doe  
3, Age, 80  
3, Zip, 80235
```

- Easy to deal with missing values



Nhận xét:

- Transactional data → convert to vector below
→ Sparsity: nhiều ô optional.
⇒ Store mostly "0" - tràn ngập bộ nhớ, nhớ nhớ nhớ em ~~~

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

TID	Bread	Coke	Milk	Beer	Diaper
1	1	1	1	0	0
2	1	0	0	1	0
3	0	1	1	1	1
4	1	0	1	1	1
5	0	1	1	0	1

Sparsity: Most entries are zero. Most baskets contain few items

Doc Id	Words
1	the, dog, follows, the, cat
2	the, cat, chases, the, cat
3	the, man, walks, the, dog

Doc Id	the	dog	follows	cat	chases	man	walks
1	2	1	1	1	0	0	0
2	2	0	0	2	1	0	0
3	1	1	0	0	0	1	1

Sparsity: Most entries are zero. Most documents contain few of the words

- **One hot encoding:** waste storage → but still use currently
 - Xử dụng trong case **(Categorical data)**
 - tùy thuộc vào độ lớn categorical mà nó sẽ phức tạp
 - ⇒ Context: sample dictrionary save

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

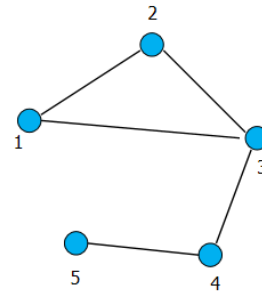
Graph Data: friend relation - brain network

- Sample: Facebook has 1billion5 users
 - Assume: each friend has maximum 500
 - have to need 1billion5 GB RAM (convert to byte, then KB)

Adjacency matrix

- Very sparse, very wasteful, but useful conceptually

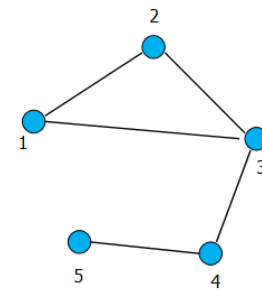
$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Adjacency list

- Not so easy to maintain

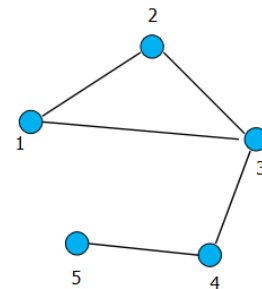
1: [2, 3]
2: [1, 3]
3: [1, 2, 4]
4: [3, 5]
5: [4]



List of pairs

- The simplest and most efficient representation

(1,2)
(2,3)
(1,3)
(3,4)
(4,5)



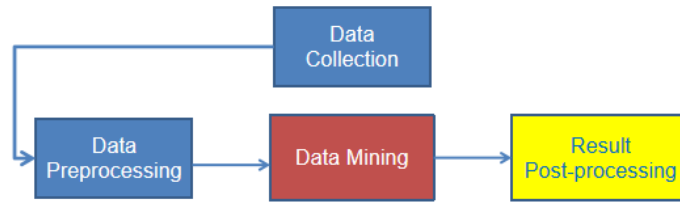
▼ D. Data pipeline & How to check the quality.



How to clean data: sometimes have to do manually - check each record to fix (Sometimes can do automation).

- **Data Collection:** crawl data through public APIs.
- **Preprocessing:** Real data is large, noisy, incomplete and inconsistent. → clean data.
 - This one is “dirty work since not take any intellectual”.
 - Affect to AI model, specific-industry decisions,...
⇒ really really important
 - Techniques:
 - **Sampling (lấy mẫu):** random 100 customers in 1000 → make sure cover the whole.
 - **Random Sampling:** choose randomly
 - **Sampling without replacement:** example - get 100 for sampling → then discard 100 from the Original data set.
 - **Sampling replacement:** sampling 100 → then add back to dataset
 - **Dimensionality Reduction:** → reduce matrix fields ⇒ keep only the significant feature
 - **Feature selection:** Selecting the most relevant features while removing irrelevant or redundant ones.
- **Post-Processing:** make data useful
 - Statistical analysis of importance of results
 - Visualization

The data analysis pipeline



- **Outlier:** exceptional data (the yellow box need to process manually)

Data Quality

- Examples of data quality problems:

- Noise and outliers
- Missing values
- Duplicate data

A mistake or a millionaire?

Missing values

Inconsistent duplicate entries

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	10000K	Yes
6	No	NULL	60K	No
7	Yes	Divorced	220K	NULL
8	No	Single	85K	Yes
9	No	Married	90K	No
9	No	Single	90K	No

▼ E. How to collect data from text? - Sample case study "Text Mining"

This one is "raw data" → how the F we can get the key?

I heard so many good things about this place so I was pretty juiced to try it. I'm from Cali and I heard Shake Shack is comparable to IN-N-OUT and I gotta say, Shake Shack wins hands down. Surprisingly, the line was short and we waited about 10 MIN. to order. I ordered a regular cheeseburger, fries and a black/white shake. So yummers. I love the location too! It's in the middle of the city and the view is breathtaking. Definitely one of my favorite places to eat in NYC.

I'm from California and I must say, Shake Shack is better than IN-N-OUT, all day, err'day.

Would I pay \$15+ for a burger here? No. But for the price point they are asking for, this is a definite bang for your buck (though for some, the opportunity cost of waiting in line might outweigh the cost savings). Thankfully, I came in before the lunch swarm descended and I ordered a shake shack (the special burger with the patty + fried cheese samp; portabella topping) and a coffee milk shake. The beef patty was very juicy and snugly packed within a soft potato roll. On the downside, I could do without the fried portabella-things, as the crispy taste conflicted with the juicy, tender burger. How does shake shack compare with in-and-out or 5-guys? I say a very close tie, and I think it comes down to personal affiliations. On the shake side, true to its name, the shake was well churned and very thick and luscious. The coffee flavor added a tangy taste and complemented the vanilla shake well. Situated in an open space in NYC, the open air sitting allows you to munch on your burger while watching people zoom by around the city. It's an oddly calming experience, or perhaps it was the food coma I was slowly falling into. Great place with food at a great price.

- Stop words: in NLP → not help anything (I you we,.... ensure the input correct grammar)
- **TF-IDF**: tìm độ "tần xuất xuất hiện" và "sự độc nhất" của đoạn text
 - The words that are best for describing a document are the ones that are **important for the document**, but also **unique to the document**.
 - **TF(w,d)**: term frequency of word w in document d
 - Number of times that the word appears in the document
 - Natural measure of **importance** of the word for the document
 - **IDF(w)**: inverse document frequency
 - Natural measure of the **uniqueness** of the word w
 - **TF-IDF(w,d) = TF(w,d) × IDF(w)**
- **Text mining**: focus on "THREE CUT":
 - **First cut**: use statistics to get the "important" word
 - **Second cut**: the 2nd layer to filter the data → filter "Stop words"
 - **Third cut**: order by **TF-IDF** → calculate the weight value, then sort by weight.
⇒ stop word - IDF(w) = 0

First cut

- Do simple processing to “normalize” the data (remove punctuation, make into lower case, clear white spaces, other?)
- Break into words, keep the most popular words

the 27514	the 16710	the 16010	the 14241
and 14508	and 9139	and 9504	and 8237
i 13088	a 8583	i 7966	a 8182
a 12152	i 8415	to 6524	i 7001
to 10672	to 7003	a 6370	to 6727
of 8702	in 5363	it 5169	of 4874
ramen 8518	it 4606	of 5159	you 4515
was 8274	of 4365	is 4519	it 4308
is 6835	is 4340	sauce 4020	is 4016
it 6802	burger 432	in 3951	was 3791
in 6402	was 4070	this 3519	pastrami 3748
for 6145	for 3441	was 3453	in 3508
but 5254	but 3284	for 3327	for 3424
that 4540	shack 3278	you 3220	sandwich 2928
you 4366	shake 3172	that 2769	that 2728
with 4181	that 3005	but 2590	but 2715
pork 4115	you 2985	food 2497	on 2247
my 3941	my 2514	Most frequent words are stop words	
this 3487	line 2389		
wait 3184	this 2242	cart 2236	not 1655
not 3016	fries 2240	chicken 2220	your 1622
we 2984	on 2204	with 2195	so 1610
at 2980	are 2142	rice 2049	have 1585
on 2922	with 2095	so 1825	

- Remove stop words
- Stop-word lists can be found online.

ramen 8572	burger 4340	sauce 4023	pastrami 3782
pork 4152	shack 3291	food 2507	sandwich 2934
wait 3195	shake 3221	cart 2239	place 1480
good 2867	line 2397	chicken 2238	good 1341
place 2361	fries 2260	rice 2052	get 1251
noodles 2279	good 1920	hot 1835	katz's 1223
uopudo 2261	burgers 1643	white 1782	just 1214
buns 2251	wait 1508	line 1755	like 1207
broth 2041	just 1412	good 1629	meat 1168
like 1902	cheese 1307	lamb 1422	one 1071
just 1896	like 1204	halal 1343	deli 984
get 1641	food 1175	just 1338	best 965
time 1613	get 1162	get 1332	go 961
one 1460	place 1159	one 1222	ticket 955
really 1437	one 1118	like 1096	food 896
go 1366	one 1012	one 1062	one 896
food 1296	Commonly used words in reviews, not so interesting		
bowl 1272			
can 1256	park 887	night 832	order 720
great 1172	can 860	time 794	pickles 699
best 1167	best 849	long 792	time 662
		people 790	

▼ F. Normalize (chuẩn hoá) và Normal Distribution (phân phối chuẩn)



- **Normalization** (1NF - BCNF): eliminates redundancy, ensures data integrity, and improves query efficiency.
→ Normalize càng cao → data càng strict
- **Normal distribution (phân phối chuẩn)**: probability distribution where most data points **cluster around the mean**, forming a symmetric **bell-shaped curve**.
 - Phân phối có hình cái chuông (E.G: Phổ điểm thi đại học).

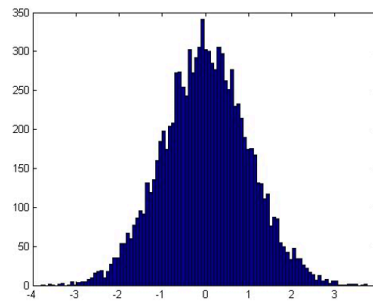


Formula:

- Probability density function (PDF) of a normal distribution.
 - là cái công thức ở dưới.
- It depends on the **mean (μ)** (center of the distribution) and **standard deviation (σ)** (spread of the data).

Normal Distribution

$$\phi(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$



This is a value **histogram**

- An important distribution that characterizes many quantities and has a central role in probabilities and statistics.
 - Appears also in the central limit theorem
- Fully characterized by the **mean μ** and standard **deviation σ**

▼ Lecture 3 - Classification (phân lớp - part 1) :



How to solve “overfitting” có thể dùng cho all context Machine Learning. ?

- Nên dùng “Grid Search” để tìm ra thông số - params tốt nhất → để tuning lại model

⇒ tìm **hyperparameters (phục vụ cho việc train mô hình)**.

Refit & Tuning Model ⇒ tìm hypermeter

⇒ “Prune” kỹ thuật tỉa bớt cây decision tree.

- Decision Trees (DTs) are a non-parametric supervised learning method used for Classification and Regression.
- **Pre-pruning (early stop)**: stop khi model được train có labels cùng lớp → không cần học lại nữa. (tránh học. quá complex)
 - Cons: có thể mất information quan trọng.
- **Post-pruning**: train hết model → sau đó prune những thứ không cần. thiết (context tree → prune node thừa).

- Phân biệt Accuracy & Precision
 - Accuracy: $TP + FN / \text{sum all}$ (the precision that model can predict)
 - Precision: just calculate only the True Positive

▼ Các mô hình phân lớp - classification (có rất nhiều model):

- **Random forest** = sum của nhiều “Decision Tree”.
- **Rule-based** (học theo “rule” để đoán).
- **Memory based** (dựa vào experience để đonas model).
- **Neural network** (ANN, CNN)

- **Naive Bayes (dựa trên toán thống kê)** - Lecture 4
- **SVM** (Support vector machine) - Lecture 4

▼ A. Classification (phân lớp) - Basic concept:



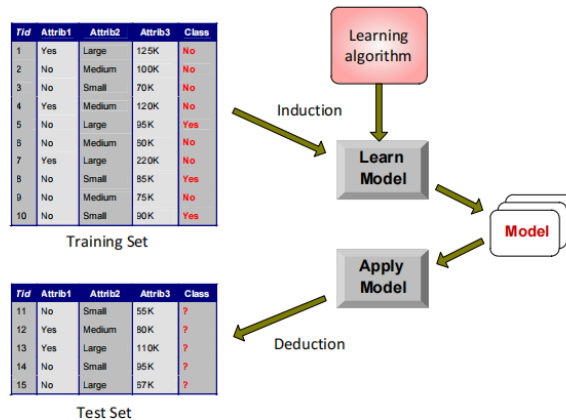
- Classification: **discriminating between different classes** of objects.
 - **Classification** is the task of **learning a target function f** that maps attribute set x to one of the predefined class labels y .
→ **target function** = classification model.
 - **Predictive modeling**: Predict unseen data.
 - **Descriptive modeling**: Explanatory tool to distinguish between objects of different classes
 - **REAL CASE STUDY**: Classifying credit card transactions as legitimate or fraudulent
 - VD: **Giả sử mất tín dụng** → người khác dùng để mua đồ (hệ thống phân tích có mua bán bất thường).
 - Trường hợp ngoại lệ: người dùng mua món hàng đó giá cao bất thường so với thông thường.
⇒ Solve: xác thực
-
- **Cách chuẩn bị data training:**
 1. "Training Set" → "known class labels" (là tập data biết trước kết quả để train)
 2. "Test set" → use to "evaluate" quality model (giống Training set, nhưng dùng để đánh giá lại mô hình).
⇒ Test set (che nhãn → để mô hình đoán).



Analogy:

- **Training Set:** Nam học IELTS 8.0 ở nhà và trung tâm.
- **Test Set:** Nam đi "evaluate" ở IDP thi được 8.0 (bản chất tập Test set IDP họ có đáp án, nhưng Nam không biết trước).

Illustrating Classification Task



• Cách train model Classification (supervised):

1. Find examples (**training set = data with labeled**)
2. Find a set of **features** that discriminate "each class from dataset" & "new example".
 - **labels = classes**
3. Create a **function (Training Model)** → extracts the features → decides the class (predict).

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical

categorical

continuous

class

This is "Label" in data

One of the attributes is the **class attribute**
In this case: Cheat

Two **class labels** (or **classes**): Yes (1), No (0)

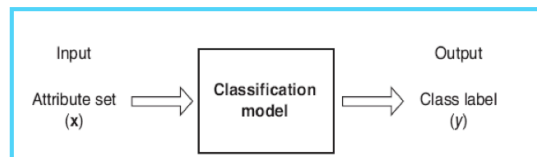


Figure 4.2. Classification as the task of mapping an input attribute set x into its class label y .

Evaluation of classification models (RẤT Quan trọng)



Why we need to use confusion matrix???

Context: guess 100 with cancer disease (10 of them has cancer)

→ model predict 100 people don't have (sound good huh =))

→ accuracy 90%

BUT.....

10 of them cancer → but it guess no one cancer, although "accuracy rate" = 90%.

⇒ Bad model

How to solve this ???

⇒ Use confusion matrix



- **Confusion matrix:**

- Predicted Class (nhãn dự đoán)
- Actual Class (nhãn thực tế)

- **Ma trận "tốt" (Main diagonal):** là ma trận có đường chéo chính đúng (Các giá trị ở bên ngoài thấp)

VD: hình ở dưới, các giá trị ở bên ngoài f_{11} và f_{00} thấp → tốt.

Giải thích ma trận dưới:

- True False: là kết quả thực tế
- Positive & Negative: is model prediction response.

f_{11} : tổng số câu dự đoán đúng - mà trong thực tế đáp án đúng

f_{00} : tổng số câu dự đoán sai - mà thực tế sai

f_{01} : tổng số câu dự đoán sai - mà thực tế đúng

f_{10} : tổng số câu dự đoán sai - mà thực tế đúng

- **Confusion matrix**

Actual Class	Predicted Class	
	Class = 1	Class = 0
Class = 1	f_{11}	f_{10}
Class = 0	f_{01}	f_{00}



Tại sao lại phải phân loại ra 2 trường hợp sai:

- Hình dung test covid
 - Mua kit test: có bị mà nói không bị (1)
 - không bị mà nói bị (2)
- ⇒ Đây là 2 trường hợp sai - nhưng trường hợp sai (1) nghiêm trọng hơn.

$$\text{Accuracy} = \frac{\# \text{ correct predictions}}{\text{total \# of predictions}} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

$$\text{Error rate} = \frac{\# \text{ wrong predictions}}{\text{total \# of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

▼ B. Decisions Tree (more 2 tree = random forest)



Random forest:

- Context: a single tree "may" too **overconfidence** and **overfitting**.
→ Same as one person vote for the idea (độc tài ghia 😊)

How to solve it?

- **Random Forest** training multiple trees independently and then combining their results. (It will observe all the features (assume 10 features - and tree will learn all)).

1. **Each tree is trained on a different random subset** of the data (Bootstrap Sampling).

→ **Randomly select samples involves selecting sample (with replacement)**, allowing the same object to be chosen multiple times

- With **Replacement** = put data back and can chose multiples time.
- Some sample may appear, why others may not.

Ex: **Given dataset:** {1, 2, 3, 4, 5}

Example of 3 different bootstrap samples:

(1, 3, 4) → Randomly

(1, 5, 1) → Notice that 1 **appears twice** (replacement allowed)

(4, 3, 5) → 2 do not appear.

→ Assume has 3 tree and train data individually.

2. **Each tree considers a random subset of features** at each split.

Example: **10 features** (F1, F2, ..., F10).

◆ **Single Decision Tree** → Looks at **all 10 features** at every split.

◆ **Random Forest** → At **each split**, randomly selects a subset of, say, **3 features** to choose the best split.

Tree 1:

- **Split 1:** Randomly considers (F2, F5, F8) → Picks the best.
- **Split 2:** Randomly considers (F1, F6, F9) → Picks the best.

Tree 2:

- **Split 1:** Randomly considers (F3, F7, F10).
- **Split 2:** Randomly considers (F2, F4, F9).

Each tree learns different **patterns**, making the **ensemble more generalizable and powerful!**

⇒ reduce tree to learn the same thing.

3. **Final prediction is based on aggregation:**

- **For classification:** Majority vote (like family vote for sunday meal 😊).
- **For regression:** Average of predictions.

Analogy to Political Voting

- **Single Decision Tree** → Like one person deciding the leader (biased).
- **Random Forest** → Like a democratic election where multiple people vote, reducing bias from any single voter.



"Decision Tree" and "Tree" on DSA - differentiate.

- **Decision Tree**: Sẽ học hết dataset và map (ánh xạ) thành cây.
 - Cài đặt Decision Tree → dùng đệ quy (thay vì if else).
 - Một dataset → có nhiều Decision Tree
- Decision tree: Đi từ node gốc trước.
 - **Branch (nhánh)**: represents an outcome of the test
 - **Internal node (node trung gian)**: kiểm tra điều kiện
→ test on an attribute
 - **Leaf nodes (node lá)**: represent class labels → để quyết định object đó nhãn gì.

Fact: không thể tìm 1 decision tree (hoàn hảo).

- Chỉ có cây "tốt hơn" và mô hình "GẦN ĐÚNG".
- Goal train model: cố gắng - generalize
→ vì k thể đoán được hết.
⇒ Nhưng model phải đủ general để hiểu hết.

Advantages Decision Tree:

- Dễ viết &. Easy to interpret (Small-sized).
- Sẽ là mô hình chạy rất tốt → nếu data tốt.
- Extremely fast → không cần phần cứng quá phức tạp

Tree induction: cây gần giống với data nhất.

- Goal: Find the tree that has **low classification error** in the training data (training error)
- Finding the **best decision tree** (lowest training error) is **NP-hard**
 - Giải pháp chung - Heuristics → "finding Better" tree and best split.
- Using greedy to find tree

- Use algo: Hunt, CART, ID3,...

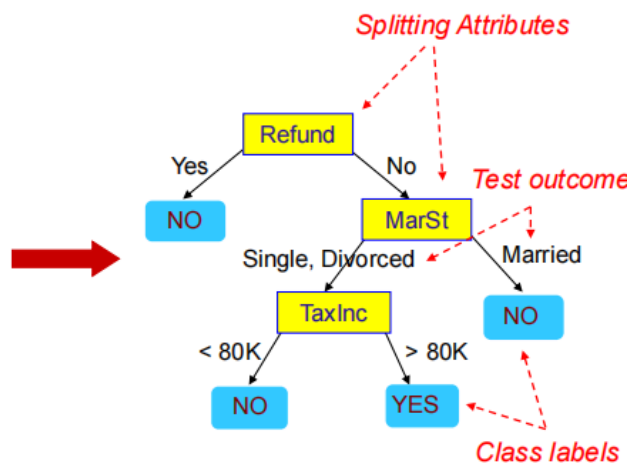
• **Trường hợp nào mà không có cây?** hình dưới đây

- Nếu ở record 11 → data giống nhau (nhưng thay cheat = No) ⇒ không thể map được Decision Tree.
- Mô hình dưới đây để **đoán "labels - cheat"**.

categorical categorical continuous class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

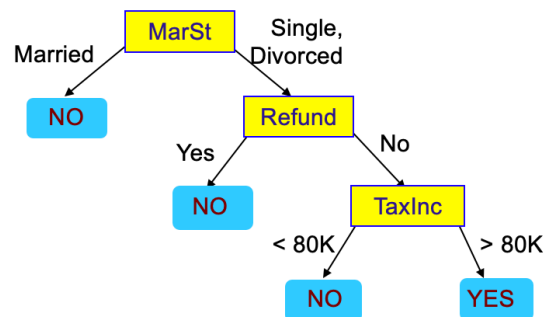
Training Data



Model: Decision Tree

categorical categorical continuous class

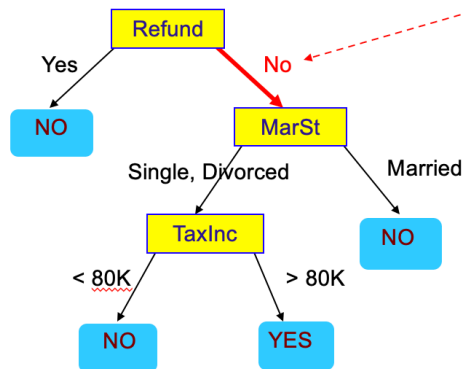
Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

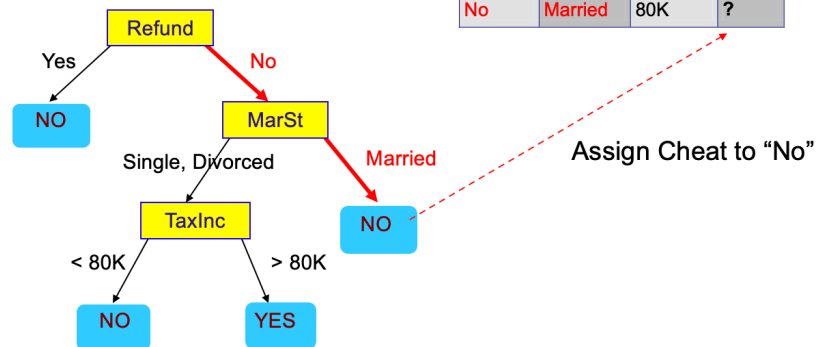
Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



▼ C .Some algorithms use for "decision tree" only:

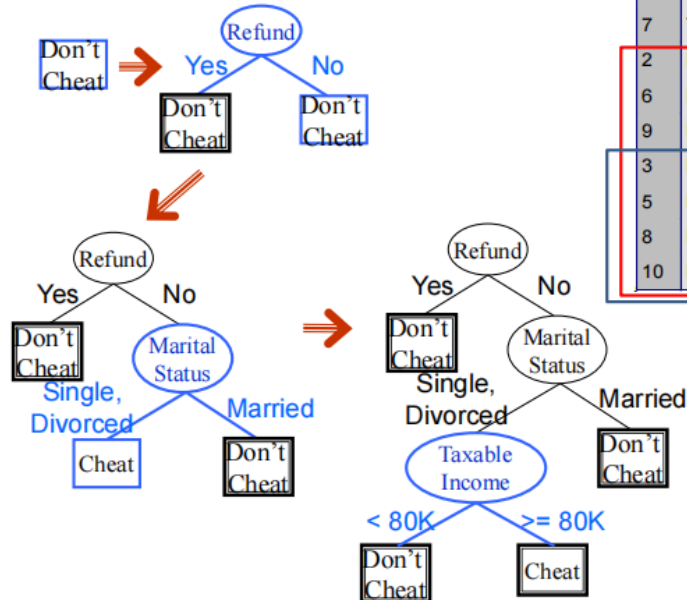
Chia data thành bao nhiêu nhánh.

Hunt's Algo:

- Explain: Xét "Refund" → nếu Yes & mặc định Cheat = No.

- Nếu “Thay đổi thứ tự xét” → ảnh hưởng đến cây

Hunt's Algorithm



	Refund	Status	Income	Cheat
1	Yes	Single	125K	No
4	Yes	Married	120K	No
7	Yes	Divorced	220K	No
2	No	Married	100K	No
6	No	Married	60K	No
9	No	Married	75K	No
3	No	Single	70K	No
5	No	Divorced	95K	Yes
8	No	Single	85K	Yes
10	No	Single	90K	Yes

▼ D. Practical issues of Classification:

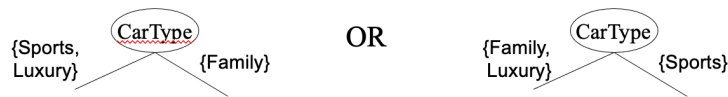
- **Under-fitting:** data not enough to build model - model too simple
Analogy: học ít → thi toang
 - Solve Under-fitting:
 - Tự tạo (**Augmented - tăng cường**): Ví dụ chụp ảnh thường không có “labels” → thì ảnh sẽ tự tạo
- **Overfitting:** data enough and learn thorough (học quá vẹt) - model too complex
Analogy: rập khuôn, học cho cố vô đối câu hỏi → lại k dùng được.
- Vấn đề phân nhánh (**Splitting**):

Splitting Based on Nominal Attributes

- **Multi-way split:** Use as many partitions as distinct values.

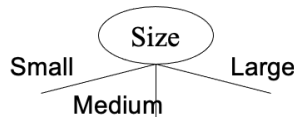


- **Binary split:** Divides values into two subsets. Need to find optimal partitioning.

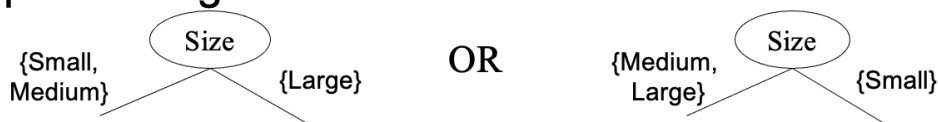


Case Study: Vấn đề về Split

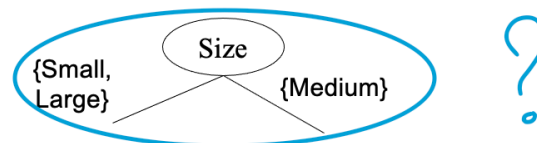
- **Multi-way split:** Use as many partitions as distinct values.



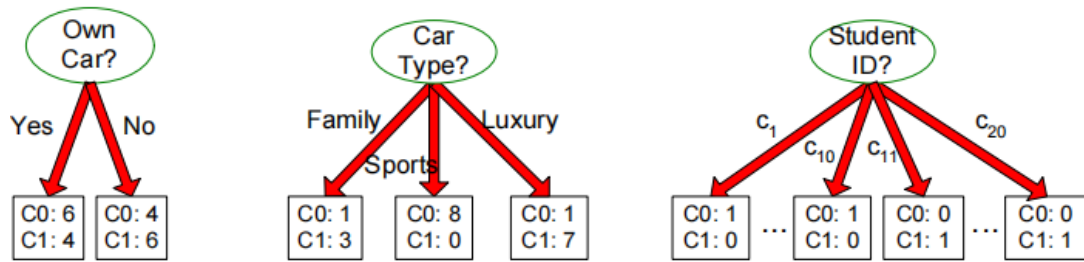
- **Binary split:** Divides values into two subsets – **respects the order**. Need to find optimal partitioning.



- What about this split?



Case Study: Vấn đề về Split trong Classification.



Lấy ví dụ: trong tay có 20 viên đậu đỏ - xanh

- **Trường hợp 1:** chia 2 nhóm (nhóm 6 đỏ - 4 xanh và 4 - 6) → giả sử bốc lên đoán (% đúng sẽ rất thấp).
- **Trường hợp 2:** chia 3 nhóm (Sẽ có nhóm 8 đỏ → giả sử bốc lên vẫn dám đoán → % đúng cao).
- **Trường hợp 3:** mô hình k thực tế → tất cả % 50 → mô hình có thể dự đoán sai.

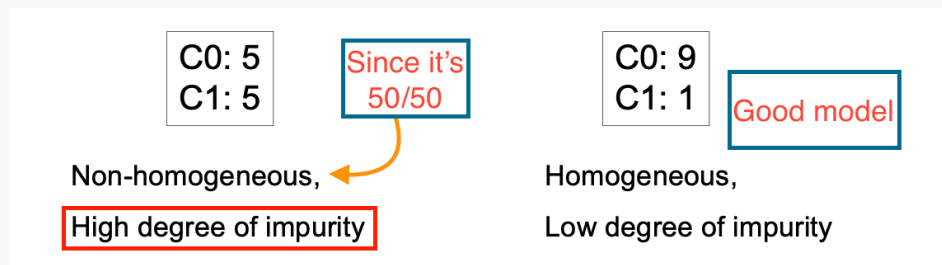
⇒ **mô hình 2**, có sự đồng nhất (thực tế)

- Lỗi sai của làm dự án:
 - Bốc một metric không liên quan đi dự đoán (lấy MSSV đoán điểm) ⇒ Sai từ đầu idea.

▼ E. How to find "best split" for Decision Tree ?



- How to determine "Best Split" tree:
 - Dùng **Greedy** approach (tìm cái tốt nhất - heuristics)
 - ⇒ Creation of nodes with **homogeneous (có tính đồng nhất)** class distribution is preferred.
- **Impurity**: mức độ mixture của data trong một node
- Need a measure of node **impurity**
 - ⇒ **Low impurity is the best**





Measuring Node Impurity: (3 thông số đo lường) → dùng để đánh giá best split của decision tree.

- $p(i|t)$: **fraction of records** associated with node t belonging to class i

1. Entropy (H) 🔥

- Measures **impurity or uncertainty** in data.
- Formula:

$$H(t) = - \sum p(i|t) \log p(i|t)$$

- **Low Entropy** → More pure node (better)
- **High Entropy** → More mixed classes (worse)
- Used in: ID3, C4.5

2. Gini Index (Gini Impurity) 🎯

- Measures **how mixed** a node is.
- Formula:

$$Gini(t) = 1 - \sum [p(i|t)]^2$$

- **Lower Gini** → Better node purity.
- Used in: CART

3. Classification Error: Measures misclassification rate decision. tree

$$\text{Classification error}(t) = 1 - \max_i [p(i|t)]$$

⇒ Gini được áp dụng nhiều trong Decision Tree.

- **Gain (Information Gain):** Measures the effectiveness of a split.
 - **Higher Gain** = Better split (reduces impurity).
 - **If impurity is entropy, the gain is called information gain.**

Gain (Information Gain, IG)

- Measures how much impurity is **reduced** after a split.
- **Formula:**

$$IG = I(parent) - \sum \frac{N(v_j)}{N} I(v_j)$$

- **Higher IG** → Better split → More pure child nodes.
- **If $I = H$ (Entropy), it is called Information Gain.**

Gain Ratio

- Normalized version of **Information Gain** to **avoid bias** towards attributes with many values.
- **Formula:**

$$Gain\ Ratio = \frac{Information\ Gain}{Split\ Info}$$

- Used in: C4.5



- **Entropy:** đo lường mức độ hỗn loạn (impurity - uncertainty) của một tập dữ liệu. Nếu một tập dữ liệu chứa nhiều loại nhãn phân loại với tần suất gần như nhau, entropy sẽ cao. Nếu tất cả các nhãn giống nhau, entropy sẽ bằng 0. → small entropy is good.
- **Gini Index:** đo mức độ thuần nhất của một tập dữ liệu (giống Entropy nhưng dễ tính toán hơn) → Faster calculate
- **Information Gain:** Helps select the best attribute for splitting
→ Helps select the **best** attribute for splitting.
- **Gain Ratio (phát triển dựa vào Information Gain):** normalizes the information gain by considering the **number of branches** a split result → reduces bias from "information gain".
 - Gain Ratio use "**Split Information**": đo độ phân tán của dữ liệu dựa trên số lượng nhánh sau khi chia. Fix Imbalance Tree.
→ Context: A decision tree just has the right side, an imbalance tree, making the tree answer on only 1 side → biased.
- **sklearn.tree.DecisionTreeClassifier** và **sklearn.tree.DecisionTreeRegressor** đều dựa trên **CART**.
- sklearn không hỗ trợ ID3, C4.5, hay C5.0 trực tiếp

▼ Phân biệt ID3, C4.0, C5.0 và CART trong Decision Tree

ID3 (Iterative Dichotomiser 3): The algorithm creates a multiway tree, Use for "Classification" and Open Source purpose.

Process:

1. Finding for each node (i.e using "Greedy approach) the **categorical feature** that has "THE BEST - **INFORMATION GAIN**". Use **Entropy**.

→ ID3 is specifically designed to work with **categorical features** as input (not numerical).

E.G: height (short, normal, tall) → ID3 is created to work with this.

1. Trees are grown to maximum size.
2. Apply "Pruning" tree to make the tree generalize to unseen data → avoid overfitting (Tỉa bớt node → tránh overfitting).

Note: ID3 does not have built-in "pruning" mechanism.

C4.5 is the successor of ID3, Use for "Classification", and Open Source purpose.

Advantages:

1. Work both numerical (continuous data) and categorical data (fix limitation from ID3). C4.5 dynamically finds the optimal split point(s) to create intervals.

- Discretize the numeric data → create the intervals, height, 1m65 - 1m7 - 1m75 - 1m8 - 1m85,.....

For example, it might determine that a good split is at 1m71, creating two branches: "Height ≤ 1m71" and "Height > 1m71".

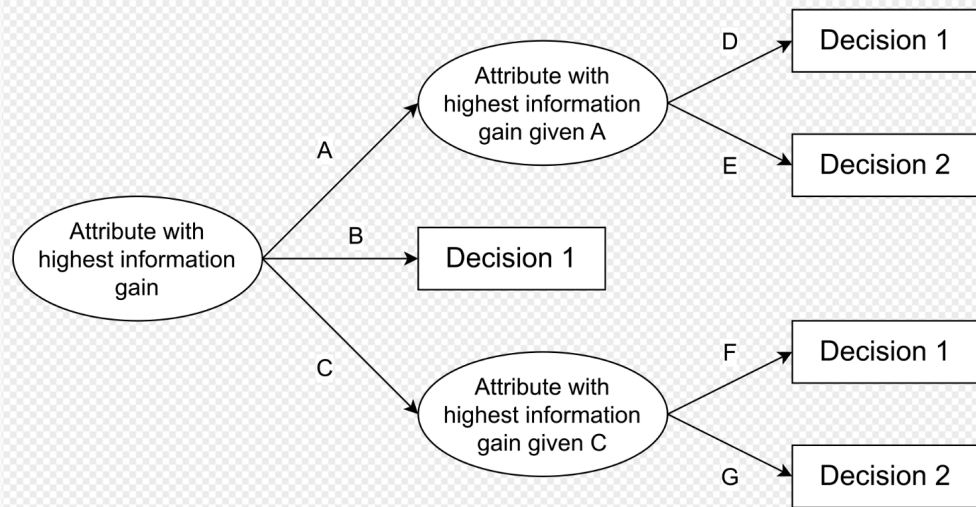
1. Have a "post-prune" mechanism after building the tree as a built-in function.
2. **Uses Gain Ratio (use Entropy to calculate Gain Ratio)**: Instead of Information Gain, C4.5 uses **Gain Ratio** as the splitting criterion → avoid biased tree (imbalanced tree, on 1 side only).
3. Handle "missing data"

C5.0 is Quinlan's latest version release under a proprietary license (not open source), next improvement from C4.5

- Run faster than C4.5 and using less memory
- Use "Entropy" to calculate "Gain Ratio"
- Include "pruning" and use for "Classification"

CART (Classification and Regression Trees): deal with numerical and categorical.

1. Use Binary Tree (ID3 C4.5 C5.0 use multiple Tree) using the feature and threshold to create "best split"
2. Use "Gini" for calculate → make this more faster compare to
3. Use for Classification and Regression + open source purpose.



Categorical Attributes

- For **binary** values split in two
- For **multivalued** attributes, for each distinct value, gather counts for each class in the dataset
 - Use the **count matrix** to make decisions

Multi-way split

	CarType		
	Family	Sports	Luxury
C1	1	2	1
C2	4	1	1
Gini	0.393		

Two-way split
(find best partition of values)

	CarType	
	{Sports, Luxury}	{Family}
C1	3	1
C2	2	4
Gini	0.400	

	CarType	
	{Sports}	{Family, Luxury}
C1	2	2
C2	1	5
Gini	0.419	

Continuous Attributes

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing impurity
 - Choose the split position that has the least impurity

		Cheat		No		No		No		Yes		Yes		Yes		No		No		No		No			
		Taxable Income																							
Sorted Values	→	60		70		75		85		90		95		100		120		125		220					
Split Positions	→	55		65		72		80		87		92		97		110		122		172		230			
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>		
Yes		0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0		
No		0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0		
Gini		0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420			

▼ F. Evaluation decision tree:



Some params use for "evaluation:

- **Accuracy (Độ chính xác):** Tỷ lệ dự đoán đúng trên tổng số mẫu.
- **Precision (Độ chính xác theo class):** Tỷ lệ mẫu được dự đoán đúng trong số các mẫu được gán vào lớp đó.
→ How many predicted positives are correct?
- **Recall (Độ bao phủ - Sensitivity):** → How many actual positives are correctly identified?
- **F1-score:** Trung bình điều hòa của Precision và Recall.

Methods Estimate model

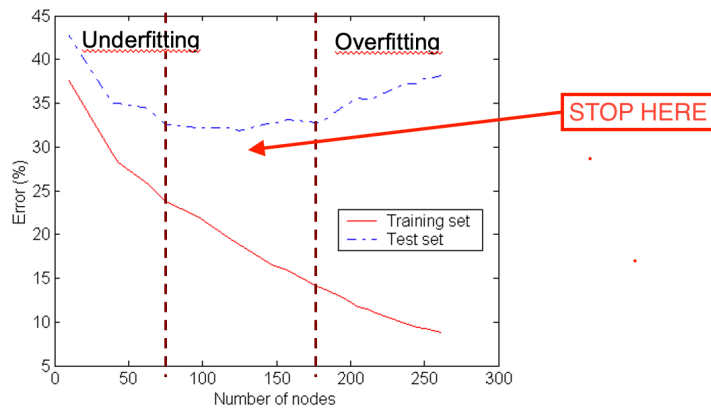
- Get random 2/3 dataset for train - the rest 1/3 for test model.

- **Cross Validation:**

- Dataset chia thành mỗi fold (Ví dụ: chia ra 10 phần)
 - Lấy 1 phần đầu test → 9 phần sau để học
 - Lấy 1 phần thứ 2 để test → 9 cái còn lại train..... cho đến khi hết 10 phần
 - ⇒ trung bình cộng sẽ ra được accuracy.

- **Generalization:** The ability of the model to predict data points that it has not already seen.

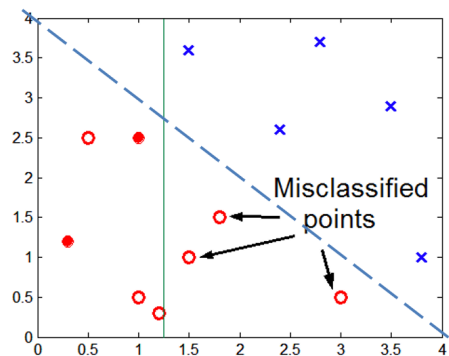
Underfitting and Overfitting



Underfitting: when model is **too simple**, both training and test errors are large

Overfitting: when model is **too complex** it models the details of the training set and fails on the test set

Overfitting due to Insufficient Examples



Lack of data points in the lower half of the diagram makes it difficult to predict correctly the class labels of that region

- Insufficient number of training records in the region causes the decision tree to predict the test examples using other training records that are irrelevant to the classification task

Phân biệt: Confusing Matrix - Cost Matrix - Weight Matrix

- Confusion Matrix → Model Evaluation (Technical Performance).

- **Confusion matrix**

		Predicted Class	
		Class = 1	Class = 0
Actual Class	Class = 1	f_{11}	f_{10}
	Class = 0	f_{01}	f_{00}

- Cost Matrix → Business Decision (Real-World Impact).
- Weight matrix: Deep learning, AI dùng nhiều

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix}$$

▼ Lecture 4 - Classification - (part 2):



- **Naïve Bayes is a type of a generative model**
 - Mô hình pick category nhãn
→ Từ nhãn "label" sinh ra thuộc tính - Generative
 - Logistic Regression and SVM are **discriminative models**
 - perform classification tốt hơn (do là trained model)
 - **Tập trung vào tìm decision boundary (là đường phân cách data (reference at 2. SVM))**
- Goal is to discriminate class of people.

Fours Algorithm for Classification:

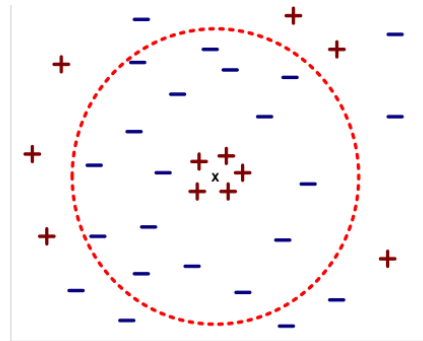
▼ 1. KNN - Nearest Neighbor Classification (cây gần nhất):



- K-Nearest Neighbors (KNN) cây láng giềng gần nhất (k - lân cận): supervised
 - K-NN là "lazy learner" → nó có thể trả ra kết quả nhanh - $O(N)$
 - KNN does not build an explicit model; it simply memorizes training data
 - **Non-Parametric: It makes no assumptions** about the underlying data distribution.
 - non-parametric and instance-based (học bằng dataset để predict)

- **K Selection (chọn số điểm K - number of neighbors):**
 - **Small K** → High variance, overfitting. (scale too small)
 - **Large K** → High bias, underfitting (Miss important info).
 - **Optimal K** → Found by tuning (e.g., cross-validation)

The value of k is the
complexity of the model

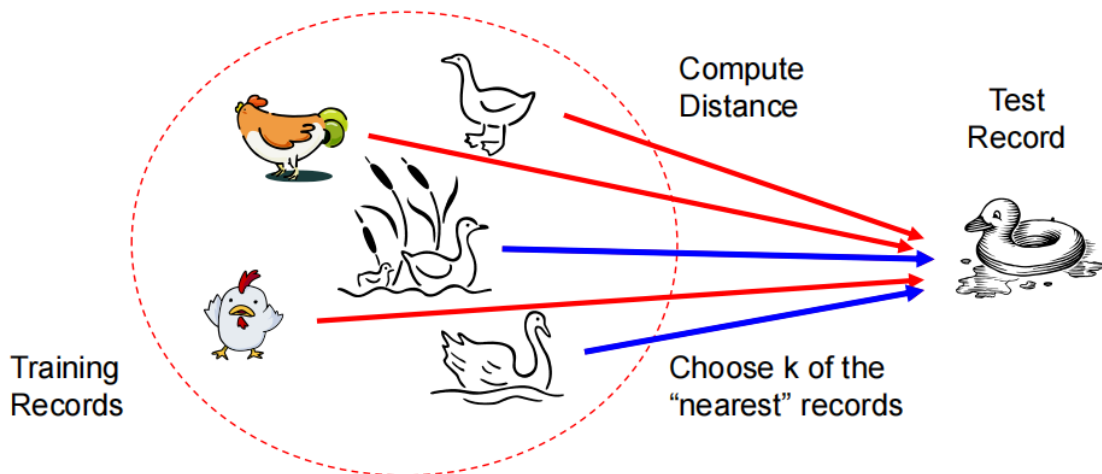




K-NN: Nearest vì tìm các data gần nhất.

Explain how it works:

1. Train data set
2. Chọn số điểm K (number of neighbors)
3. Sau đó "test record sẽ chọn k điểm gần nhất (mũi tên xanh) → xét trên "k" điểm đã chọn
4. Tính Distance (bản chất lưu bằng vector)
5. Con vịt tương đồng với 2 data được train (dựa vào số lượng K điểm) → từ đó suy ra vịt
→ Sort các "distance" tìm cái nào gần nhất ⇒ giống nhất



Requires three things

- The set of **stored records**
- **Distance Metric** to compute distance between records
- The value of **k**, the number of **nearest neighbors** to retrieve

- **Distance metric:** sẽ là khoảng cách 2 điểm (KNN sẽ hoạt động tốt dựa vào hàm số này)
⇒ tìm độ phù hợp → tức đi tìm khoảng cách của 2 kết quả cần đoán

- Euclidean distance

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

- **Scaling issues (vấn đề scale số liệu)**

- Solve: Scale Data & Normalize

Giả sử: chiều cao người từ 1m5 - 1m8 → sẽ scale dữ liệu (tránh tính toán khoảng cách sai sót)

⇒ reduce "dominant disparity (sự chênh lệch)".

- **Problem with Euclidean measure:**

- High dimensional data
 - curse of dimensionality
- Can produce counter-intuitive results

1 1 1 1 1 1 1 1 1 1 0	vs	1 0 0 0 0 0 0 0 0 0 0
0 1 1 1 1 1 1 1 1 1 1		0 0 0 0 0 0 0 0 0 0 1
d = 1.4142		d = 1.4142

- ◆ Solution: Normalize the vectors to unit length

▼ 2. Support Vector Machine (SVM):

<https://www.youtube.com/watch?v=LPI8Tw1AMrE>



Pros:

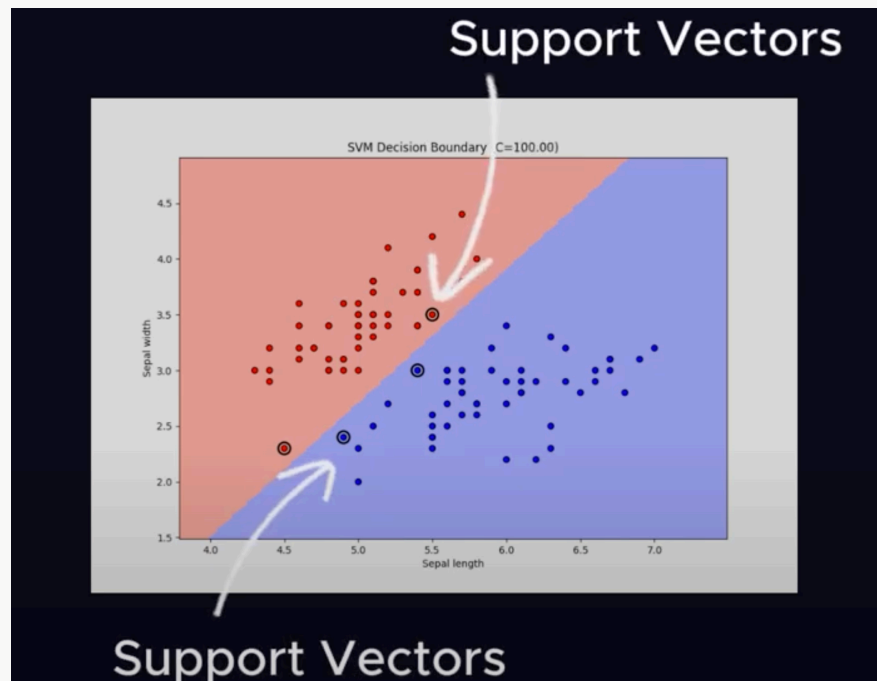
- Process well on handle "high-dimensional" data.
- Can process non-linear
- Hardly to overfit $\Rightarrow$ best at "Generalize"

$\Rightarrow$ 🤔

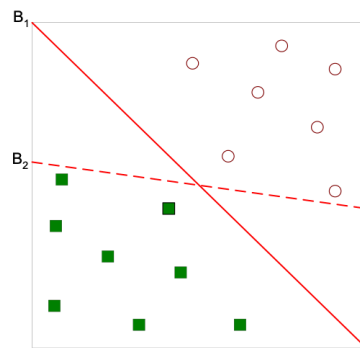


Goal: Find a linear hyperplane (**decision boundary = hyperlane**) that will separate the data and Find mô hình "tốt hơn".

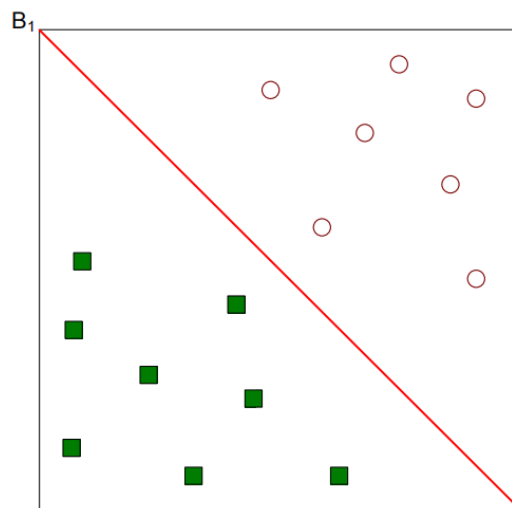
- Goal 1: "MAXIMIZE" được margin $\rightarrow$ tăng generalize.
- Actually, more. than one decision boundary.
- Support Vector - decide where that border exist



• •



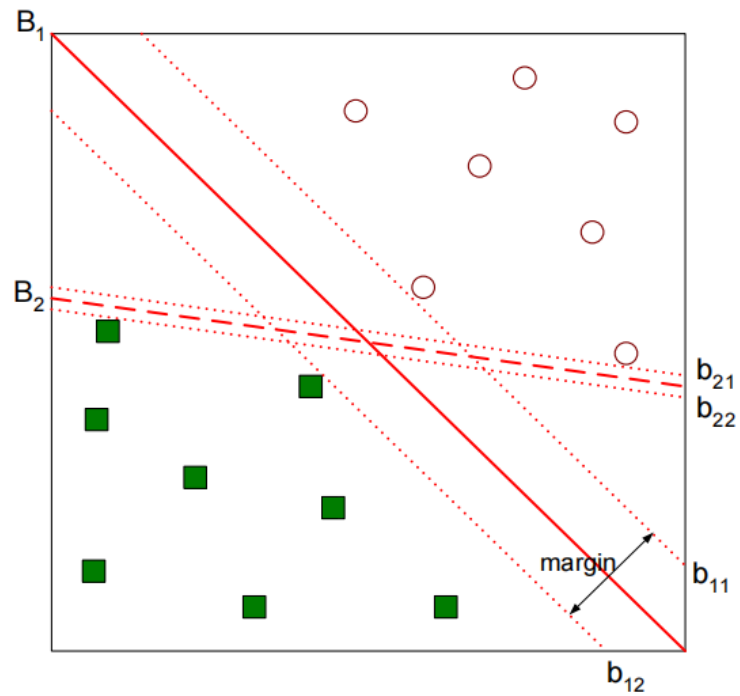
- Which one is better? B1 or B2?
- How do you define better?



Tìm một đường thẳng linear

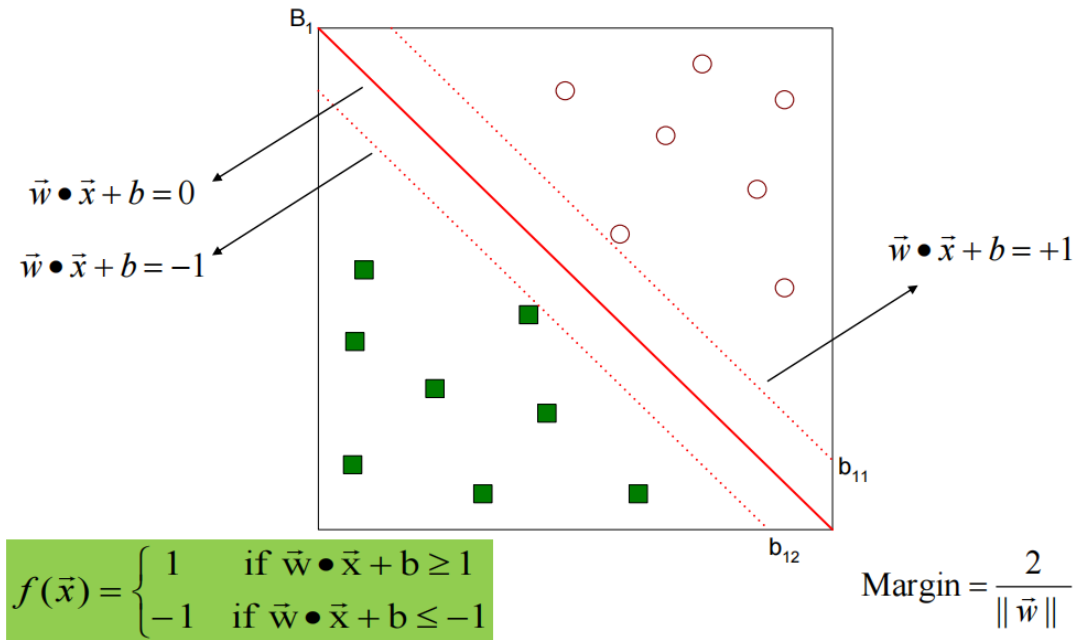
- Ví dụ: $y = ax + by + c = 0$
Giải sự cho một tọa độ $(x, y) \rightarrow$ thay vào bài toán
 $\Rightarrow$ nếu $> 0 \rightarrow$ thì ở trên đường thẳng
 $\Rightarrow$ nếu $< 0 \rightarrow$ ở dưới đường thẳng.

- **Mô hình "tốt"**: biên **margin rộng** $\rightarrow$ generalize tốt (có thể xử lý data unseen tốt).



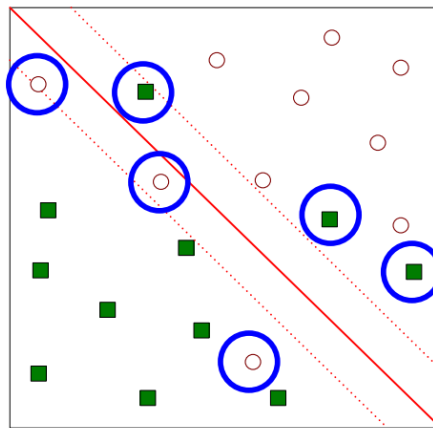
- Find hyperplane **maximizes** the margin $\Rightarrow$ B1 is better than B2

- 1 và 1: là khoảng cách margin với đường thẳng
 - giả sử: -1 = biến c nào đó $\Rightarrow$ c chia vế phải sẽ có kết quả



Bản chất:

- Giống như decision tree $\Rightarrow$ sẽ có trường hợp không cover hết các trường hợp của cây.
- Trường hợp ở dưới đây: mình sẽ bỏ qua - dùng thuật toán (một số điểm ô trong xanh bị vi phạm).
 - Các cái điểm này gọi là **"Slack variable"**



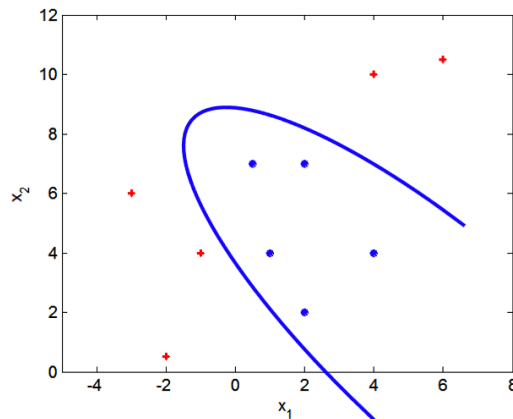


Bản chất: model luôn cố gắng đưa về dạng "TUYẾN TÍNH"

- Vì hàm tuyến tính dễ xử lý.
- Thực tế: ngoài đời toàn data và model là non linear $\Rightarrow$ chuyển về linear
 - $\Rightarrow$ **kernel trick** to transform it into a higher-dimensional space where a hyperplane can separate the classes (kernel trick nắn lại đường thẳng).
- Fact: **RẤT ÍT CÁC HÀM có thể dùng "KERNEL TRICK" $\rightarrow$ vì phải chuyển đổi từ hình 1 sang hình 2 cần tính toán thêm tham số rất nhiều**

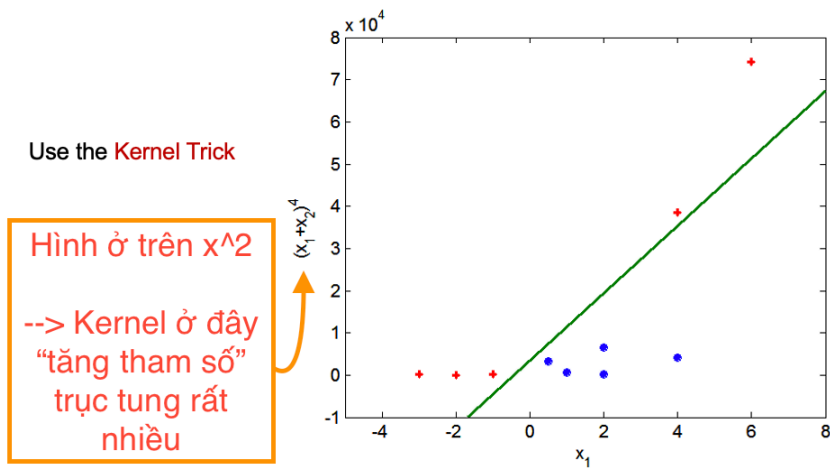
Nonlinear Support Vector Machines

- What if decision boundary is not linear?



- cố gắng ánh xạ từ hàm x_1 ở trên $\rightarrow$ sang hình ở dưới
 - Túc ánh xạ từ hình 1 $\rightarrow$ sang hình 2 (bản chất 2 hình vẫn như nhau) $\Rightarrow$ nhưng chuyển được từ non-linear $\rightarrow$ linear

- Transform data into higher dimensional space



Summary Key Components of SVM

1. **Hyperplane** – The decision boundary that separates classes.
2. **Support Vectors** – Data points closest to the hyperplane, which influence its position.
3. **Margin** – The distance between the hyperplane and the nearest support vectors.
4. **Kernel Trick** – A mathematical function that allows SVM to classify non-linearly separable data by transforming it into a higher-dimensional space.

Types of SVM

- **Linear SVM:** When data is linearly separable. (mô hình đẹp nhất)
- **Non-Linear SVM:** Uses kernels (e.g., polynomial, radial basis function - RBF) to separate complex datasets.

▼ 3. Logistic Regression - hay vl =)))



Logistic Regression:

- Instead of predicting the class of an record we want to **predict the probability of the class** (tính toán phần trăm của nhãn - thay vì đoán label) given the record
- Bản chất "Linear Regression" → tính toán từ $[0, 1]$
- The problem of predicting continuous values is called **regression problem**

Classification via regression

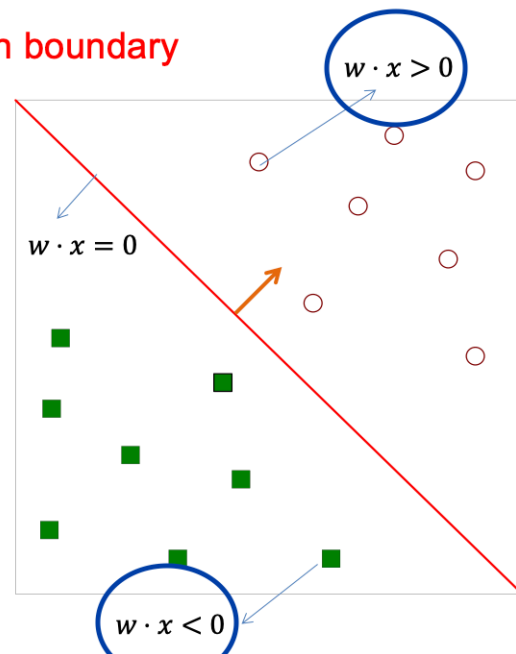
- Assume a **linear classification boundary**

For the positive class the **bigger** the **value of $w \cdot x$** , the further the point is from the classification boundary, the higher our **certainty** for the membership to the **positive class**

- Define $P(C_+|x)$ as an **increasing** function of $w \cdot x$

For the negative class the **smaller** the **value of $w \cdot x$** , the further the point is from the classification boundary, the higher our **certainty** for the membership to the **negative class**

- Define $P(C_-|x)$ as a **decreasing** function of $w \cdot x$





- Fact: các hàm không thể là "linear" $\Rightarrow$ bởi vì dữ liệu không thể match được với hàm linear.

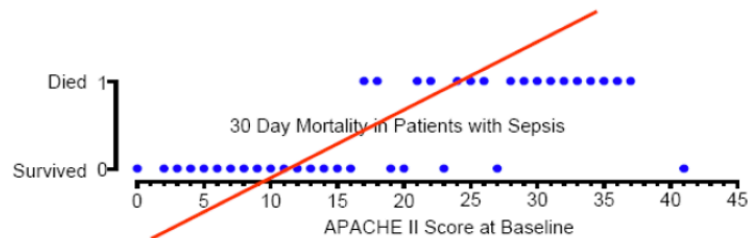
$\Rightarrow$ Nên phải dùng các hàm mũ như "logistic - log"

- Requirement: phải có một hàm số làm sao cho nó **"fit" nhất với dữ liệu**

$\rightarrow$ giả sử hàm "e mũ x" $\rightarrow$ mình sẽ cố gắng chuyển về dạng linear (bằng cách thay hàm số linear vào trong đó).

Linear regression

- A linear function is not good
 - It may produce negative probabilities, or probabilities that are greater than 1.



Thế vào "t" là một hàm tuyến tính

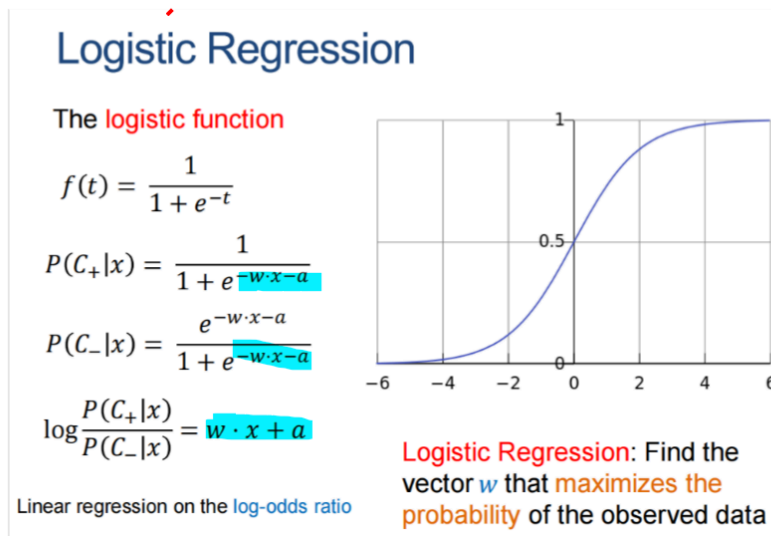
Logistic Regression: Tính ra xác suất vào từng loại 'label' thay vì đoán nhãn

Ở dưới đây: $p(c-)$: giả sử c- tức là một nhãn A

$p(c+)$: tức xác suất nhãn B

Đơn thuần: mình sẽ so sánh 2 khả năng hàm số cho nhau (từ 0 đến 1)

=> nhưng xin hơn, mình sẽ tính hàm log để tính xác suất



- **Turning point** is where the probability is **0.5**, meaning it's the threshold for decision-making.

α (Alpha) – Controls the Position of the Turning Point

- Alpha **shifts** the logistic curve **left or right** on the x-axis.
- A **smaller (more negative) alpha** moves the curve to the right.
- A **larger alpha** moves it to the left.

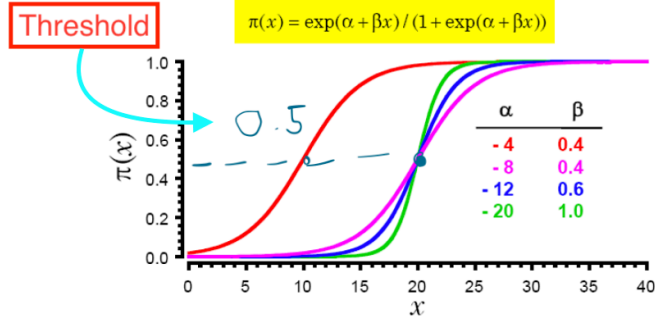
β (Beta) – Controls the Slope (Steepness)

- A **larger β** makes the curve steeper (rapid transition between 0 and 1).
- A **smaller β** makes the curve more gradual (smoother transition).

The logistic function

β controls the slope

α controls the position of the turning point



When $x = -\alpha / \beta$, $\alpha + \beta x = 0$ and hence $\pi(x) = 1 / (1 + 1) = 0.5$

Jeff Howbert Introduction to Machine Learning Winter 2012 13

The **turning point is the midpoint** of the logistic curve, where the function value is 0.5 (50% probability).

- It occurs when:

$$x = -\frac{\alpha}{\beta}$$

- At this point,

$$\alpha + \beta x = 0$$

so the logistic function simplifies to:

$$\pi(x) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

This is the **threshold** where classification decisions are usually made (e.g., if $\pi(x) > 0.5$, classify as 1, otherwise classify as 0).

▼ 4. Naïve Bayes Classifier (là một trong nhóm Generative model) - tham khảo sau nếu cần:



Bayes Classifier is a probabilistic classification technique (phân loại xác suất) based on Bayes' Theorem, which calculates the probability of a class given the input features.

Bayes Theorem Formula

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

- $P(A|B)$ = Probability of **class A** given data **B** (posterior probability).
- $P(B|A)$ = Probability of **data B** occurring given class A.
- $P(A)$ = Prior probability of class A.
- $P(B)$ = Total probability of data B occurring.

Xác suất - lớp A xuất hiện

Xác suất dữ liệu B xuất hiện nhân A (context supervised learning).

Pros:

- Kháng nhiễu - Isolated noise points
- Giảm mất mát - Missing values
- Loại attributes không liên quan - Irrelevant attributes
- Giả định độc lập - Independence assumption

Context: có 2 nhãn

- Xác suất thống kê: dựa vào định lý Bayes (dự đoán xác suất nó dính vào nhãn A và nhãn B).
- C là class
- A là tập Attributes

Bayesian Classifiers

- Approach:
 - compute the posterior probability $P(C | A_1, A_2, \dots, A_n)$ using the Bayes theorem

$$P(C | A_1 A_2 \dots A_n) = \frac{P(A_1 A_2 \dots A_n | C) P(C)}{P(A_1 A_2 \dots A_n)}$$

- Maximizing $P(C | A_1, A_2, \dots, A_n)$ is equivalent to maximizing $P(A_1, A_2, \dots, A_n | C) P(C)$
 - The value $P(A_1, \dots, A_n)$ is the same for all values of C .
- How to estimate $P(A_1, A_2, \dots, A_n | C)$?

Types of Bayes Classifiers

Type	Description	Example
Naïve Bayes	Assumes independence between features. Fast & simple.	Spam detection (words in an email are treated as independent) 📧
Gaussian Naïve Bayes	Assumes normal distribution of numerical data.	Predicting customer behavior 📊
Multinomial Naïve Bayes	Used for count-based data (e.g., word frequencies).	Text classification 📖
Bernoulli Naïve Bayes	Used for binary data (0/1, True/False).	Sentiment analysis 😊😞

▼ 5. Supervised Learning:

- **Supervised learning**: learn a model from the data using labeled data.
 - Classification and Regression,....

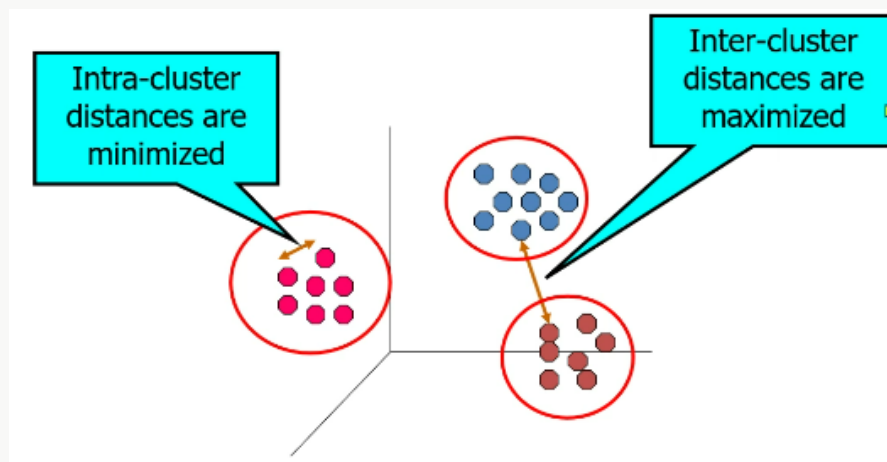
- **Unsupervised Learning**: learn a model – extract structure from unlabeled data.
→ tự học để có 1 structure tự predict.
 - Cluster, Association Rule (mẫu phổ biến kết hợp),....
- **Semi-supervised Learning**: learn a model for the data using both labeled and unlabeled data.

▼ **Lecture 5 - Clustering (phân cụm):**
▼ **A. What is Clustering:**

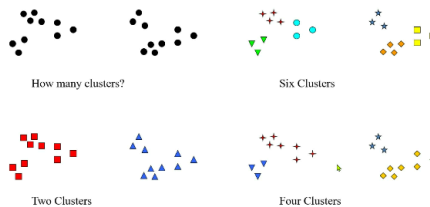


Gom thành một cụm (cluster)

- **Áp dụng trong chứng khoán - phân tích cụm ví dụ (Trump đánh thuế Việt Nam 3/4/2025 → dẫn theo cổ phiếu Việt Nam giảm) ⇒ Phân tích theo dữ liệu quá khứ.**
- Gom cụm như nào hợp lý?
 - tính các khoảng cách tương đồng nhau (**Intra-cluster** ở khoảng cách nhỏ).
Đo lường sự gắn kết/tương đồng *bên trong* một cụm
 - ⇒ Khoảng cách các cụm là (**Inter cluster**)
Đo lường sự tách biệt *giữa* các cụm khác nhau. Mục tiêu: **Càng lớn càng tốt.**



Notion of a Cluster can be Ambiguous





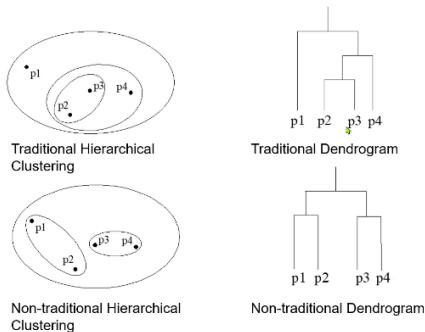
Gom cụm: chỉ có một “Đáp án” → ví dụ như hình ở dưới (cho 1 dãy data) ⇒

AMBIGUOUS

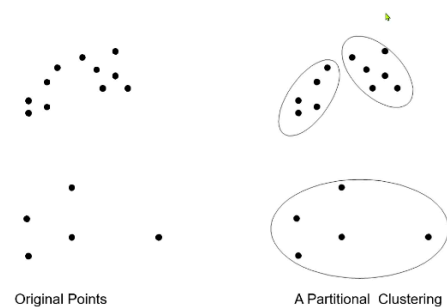
- Tuy nhiên, giả sử data chỉ có phân 2 cụm - cho dù gom thành nhiều cụm lẻ (không giải quyết được bài toán thực tế).
- Chung quy lại, tùy vào bài toán mà phân vùng clustering
-

- **Clustering is a set of clusters**
- Có hai loại phân cụm
 - **Hierarchical**: Phân chia thành “cây phân cấp
 - **Partitional**: phân chia thành các phần nhỏ

Hierarchical Clustering

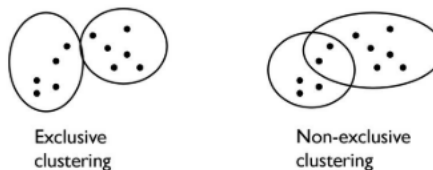


Partitional Clustering



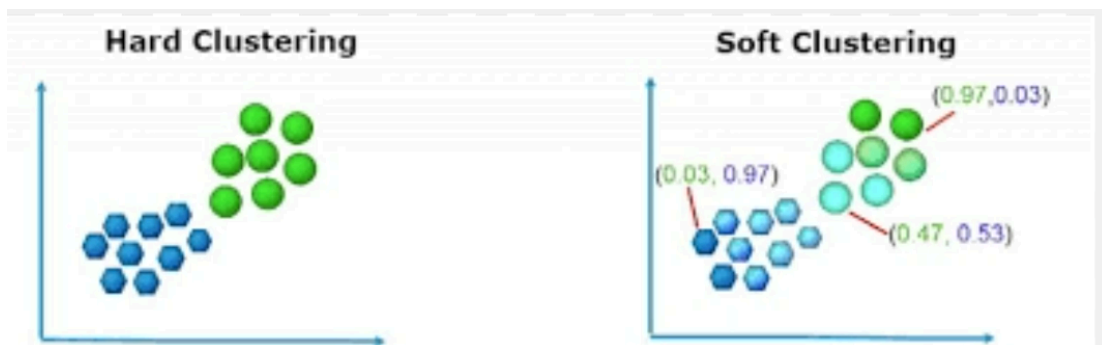
Other types of clustering

- **Exclusive (or non-overlapping)** versus **non-exclusive (or overlapping)**
 - In non-exclusive clusterings, points may belong to multiple clusters.
 - Points that belong to multiple classes, or 'border' points
- **Fuzzy (or soft)** versus **non-fuzzy (or hard)**
 - In fuzzy clustering, a point belongs to every cluster with some weight between 0 and 1
 - Weights usually must sum to 1 (often interpreted as **probabilities**)
- **Partial** versus **complete**
 - In some cases, we only want to cluster some of the data



Other type clustering:

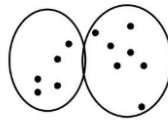
- Exclusive (non-overlapping)
- Fuzzy (gom cụm mờ): Giả sử gom 2 cụm 0 và 1 $\rightarrow$ Fuzzy có các point xen giữa giá trị 0 và 1 (Ví dụ 0.5)



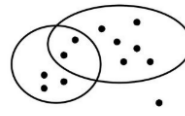
- Partial clustering: có thể cho một điểm không theo cụm nào (xét là nhiễu)

Partial versus complete

- In some cases, we only want to cluster some of the data
- e.g. to eliminate the outliers.



Complete
clustering



Partial clustering



Mục tiêu phân cụm - Clustering objective

- Well-separated: phân rõ ràng
 - Center-base (hình dung các cụm - để có tâm "Centroid" → Thuộc nhóm NP hard.
 - Contiguous Cluster (Nearest neighbor or Transitive)
 - **Density-based**
 - Conceptual (theo requirements của người đó)
-

Contiguous Cluster (Cụm lân cận):

- **Tưởng tượng:** Bạn có một đám đông người đang tụ tập.
- **Giải thích đơn giản:** Các cụm lân cận là những nhóm người đứng gần nhau, tạo thành những "vùng" riêng biệt trong đám đông.
- **Trong data mining:** Các điểm dữ liệu được gom vào cùng một cụm nếu chúng "gần" nhau trong không gian dữ liệu. Sự "gần" này thường được đo bằng khoảng cách (ví dụ: khoảng cách Euclidean).
- **Ví dụ:** Các thuật toán như K-Means thường tạo ra các cụm lân cận, nơi mỗi điểm dữ liệu thuộc về cụm có trung tâm gần nhất.

Density-based Clustering (Phân cụm dựa trên mật độ):

- **Tưởng tượng:** Vẫn là đám đông người.
- **Giải thích đơn giản:** Các cụm dựa trên mật độ là những khu vực trong đám đông mà người tập trung rất đông. Những khu vực thưa thớt người sẽ được coi là khoảng trống giữa các cụm.
- **Trong data mining:** Các cụm được xác định là những vùng có mật độ điểm dữ liệu cao, được phân tách bởi những vùng có mật độ thấp.
- **Ưu điểm:** Phương pháp này có thể tìm ra các cụm có hình dạng bất kỳ (không nhất thiết phải hình tròn như K-Means) và có thể loại bỏ các điểm nhiễu (outlier) vì chúng thường nằm trong vùng có mật độ thấp.
- **Ví dụ:** Thuật toán DBSCAN là một ví dụ điển hình của phương pháp phân cụm dựa trên mật độ. Nó xác định các cụm dựa trên số lượng điểm dữ liệu tối thiểu cần thiết trong một vùng lân cận nhất định.

Tóm lại:

- **Contiguous Cluster:** Nhóm các điểm dữ liệu dựa trên sự "gần gũi" về khoảng cách.

- **Density-based Clustering:** Nhóm các điểm dữ liệu dựa trên mật độ tập trung của chúng.

Hy vọng giải thích này dễ hiểu với bạn!

Clustering objectives

- **Center-based**
 - A cluster is a set of objects such that an object in a cluster is **closer** (more **similar**) to the “center” of a cluster, than to the center of any other cluster
 - The center of a cluster is often a **centroid**, the minimizer of distances from all the points in the cluster, or a **medoid**, the most “representative” point of a cluster



4 center-based clusters

▼ B. K - Mean algorithm:



Ví dụ về K-mean hoạt động: Bản chất tính khoảng cách các điểm và centroid để tìm cụm.

- Tính các điểm với tâm, điểm nào có kc gần $\rightarrow$ thuộc cụm đó (Ví dụ 1,1 có $d = 0 \rightarrow$ là cụm 1)
- Sau đó, tính lại khoảng cách các điểm với centroid mới (kiểm tra xem nó còn ở cụm đó hay không).

Let's imagine a tiny dataset with 5 points in 2D space, $k = 2$, to illustrate how the algorithm works:

- Dataset X : $[(1, 1), (2, 1), (4, 3), (5, 3), (5, 4)]$, so $n = 5, d = 2$.
- $k = 2$, `max_iterations = 10`.

Step 1: Initialize

- Randomly select 2 points as initial centroids, e.g., $C_1 = (1, 1), C_2 = (4, 3)$.

Step 2: First Iteration

• Assignment Step:

- For $(1, 1)$: Distance to $C_1 = 0$, to $C_2 = \sqrt{(4-1)^2 + (3-1)^2} = \sqrt{13} \approx 3.6$. Assign to cluster 1.
- For $(2, 1)$: Distance to $C_1 = 1$, to $C_2 \approx 3.2$. Assign to cluster 1.
- For $(4, 3)$: Distance to $C_1 \approx 3.6$, to $C_2 = 0$. Assign to cluster 2.
- For $(5, 3)$: Distance to $C_1 \approx 4.5$, to $C_2 = 1$. Assign to cluster 2.
- For $(5, 4)$: Distance to $C_1 \approx 5$, to $C_2 \approx 1.4$. Assign to cluster 2.
- Clusters: Cluster 1 = $[(1, 1), (2, 1)]$, Cluster 2 = $[(4, 3), (5, 3), (5, 4)]$.

• Update Step:

- New C_1 : Mean of Cluster 1 = $\frac{(1,1)+(2,1)}{2} = (1.5, 1)$.
- New C_2 : Mean of Cluster 2 = $\frac{(4,3)+(5,3)+(5,4)}{3} = (4.67, 3.33)$.

Step 3: Second Iteration

• Assignment Step (using new centroids):

- For $(1, 1)$: Distance to $C_1 = 0.5$, to $C_2 \approx 4.2$. Cluster 1.
- For $(2, 1)$: Distance to $C_1 = 0.5$, to $C_2 \approx 3.8$. Cluster 1.
- For $(4, 3)$: Distance to $C_1 \approx 3.2$, to $C_2 \approx 0.7$. Cluster 2.
- For $(5, 3)$: Distance to $C_1 \approx 4$, to $C_2 \approx 0.5$. Cluster 2.
- For $(5, 4)$: Distance to $C_1 \approx 4.5$, to $C_2 \approx 0.7$. Cluster 2.
- Clusters: Same as before (no change in assignments).



Step 3: Second Iteration

- **Assignment Step** (using new centroids):
 - For (1, 1): Distance to $C_1 = 0.5$, to $C_2 \approx 4.2$. Cluster 1.
 - For (2, 1): Distance to $C_1 = 0.5$, to $C_2 \approx 3.8$. Cluster 1.
 - For (4, 3): Distance to $C_1 \approx 3.2$, to $C_2 \approx 0.7$. Cluster 2.
 - For (5, 3): Distance to $C_1 \approx 4$, to $C_2 \approx 0.5$. Cluster 2.
 - For (5, 4): Distance to $C_1 \approx 4.5$, to $C_2 \approx 0.7$. Cluster 2.
 - Clusters: Same as before (no change in assignments).
- **Update Step:**
 - C_1 : Still (1.5, 1).
 - C_2 : Still (4.67, 3.33).

Step 4: Convergence

- Centroids didn't change, so the algorithm converges.
- **Output:**
 - Cluster assignments: $[(1, 1) \rightarrow 1, (2, 1) \rightarrow 1, (4, 3) \rightarrow 2, (5, 3) \rightarrow 2, (5, 4) \rightarrow 2]$.
 - Final centroids: $C_1 = (1.5, 1)$, $C_2 = (4.67, 3.33)$.

Pseudo code - K Means:

Input:

- X: Dataset (một tập n - data points, each with d dimensions)
- k: Số Cụm đã được xác định
- max_iterations: số lượng vòng lặp k-mean
- tolerance: Convergence threshold - mức hội tụ (optional - để thuật toán có thể dừng trước nếu đến cụm không còn biến đổi)

Output:

- Cluster assignments for each data point (các cụm cluster)
- Final centroids of the k clusters (điểm trung tâm của k cụm - tức centroid k không biến đổi nữa).

Thuật toán K-Means:

1. Initialize (khởi tạo):

- Chọn random k điểm để khởi tạo centroid (Randomly select k data points from X as initial centroids ($C_1, C_2, \dots, C_k$))
- Khởi tạo mảng cluster để lưu khoảng cách - (Initialize cluster assignments)

for each data point as empty)

2. Repeat until convergence or max_iterations reached: → (lặp cho đến khi done max iterate hoặc đủ hội tụ cụm)

a. Assignment Step: # bước này để tính khoảng cách của điểm với centroid

- For each data point x_i in X : (tính khoảng cách Euclid giữa các điểm -- > mục đích để gom các điểm với các centroid)

- Compute the Euclidean distance from x_i to each centroid C_j ($j = 1$ to

k) - (tính khoảng cách của từng point với centroid, dùng cho việc phân cụm)

- Assign x_i to the cluster with the nearest centroid:

$cluster[x_i] = \operatorname{argmin}_j (\text{distance}(x_i, C_j))$

b. Update Step: # Bước này để tính lại centroid trung bình - sau khi tìm cụm ở trên

- For each cluster j ($j = 1$ to k): #chạy số lại lần các cụm

- If cluster j is not empty:

- Compute the new centroid C_j as the mean of all points assigned to cluster j :

$C_j = (\text{sum of points in cluster } j) / (\text{number of points in cluster } j)$

- If cluster j is empty:

- Reinitialize C_j by randomly selecting a data point from X

c. Check for Convergence (optional): # nếu centroid ở bước update không còn thay đổi thì xuống bước này

- Compute the total change in centroids (e.g., sum of distances between old and new centroids)

- If change < tolerance:

- Break the loop (converged)

3. Return:

- Cluster assignments for each data point #cụm nào ở từng giá trị nào

- Final centroids ($C_1, C_2, \dots, C_k$) #trả về giá trị của cụm sau khi gom

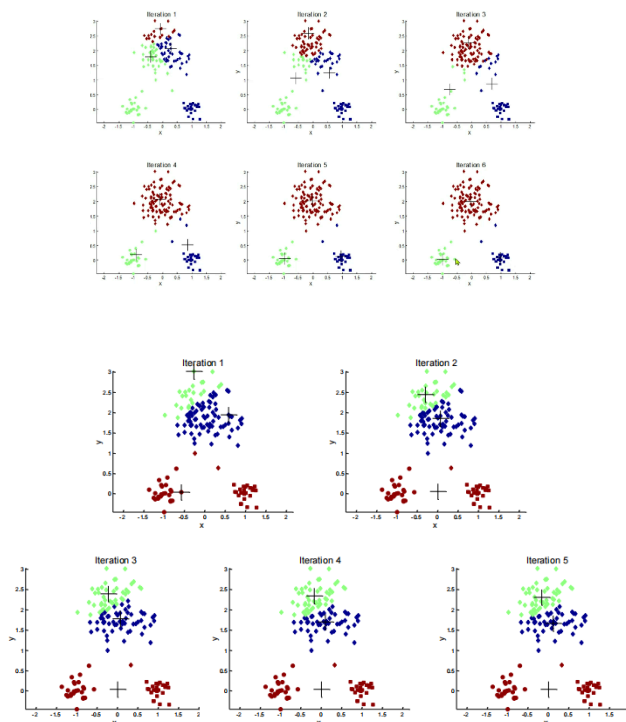


- Partitional clustering (thuộc nhóm phân chia theo phần)
- Mỗi cluster có "centroid - center point".
- Object → tìm K "Centroid và tìm xung quanh tâm" → những điểm nào ở gần tâm → gom thành một cụm
 - Nhưng fact: centroid không biết trước :DD

Dùng Lloyd's Algo - gom cụm

- Mục tiêu: do chưa biết "centroid" và "cluster" → nó chọn random ban đầu → và lọc random đến khi thành 3 cụm riêng lẻ.
 - Chọn ngẫu nhiên → gom thành tâm → tính lại tâm → gom lại → Cho đến khi không đổi tâm nữa. (Lúc đầu chạy random)
 - Kết quả: mỗi tâm nó về giữa cụm.
 - **Nhưng có hạn chế: vì mỗi lần chạy "Random"**

Importance of Choosing Initial Centroids





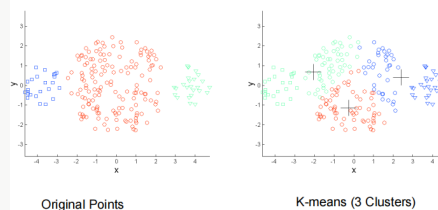
Cách giải quyết randomness ở trên:

- Có thể chạy "multiple runs" → chỉ có hy vọng những lần sau ra kết quả tốt hơn.

K-mean

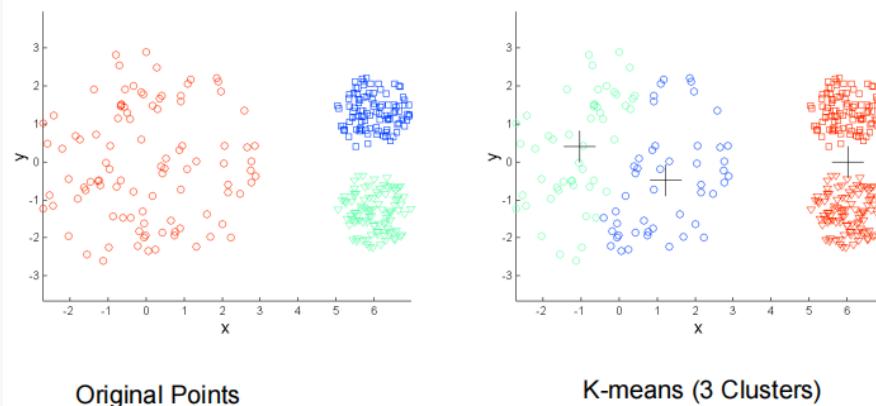
- Centroid phụ thuộc vào khoảng cách
 - mean của "euclidean"
 - median của manhattan
- Bản chất gom cụm (xung quanh tâm) - nhưng nó có một số limit
 - Khác size

Limitations of K-means: Differing Sizes



- Khác density (mật độ)

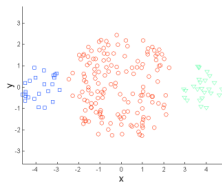
Limitations of K-means: Differing Density



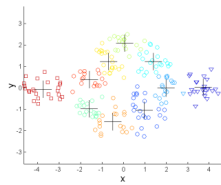
- non globular

Giải quyết các limit $\rightarrow$ tăng nhiều K $\Rightarrow$ mục tiêu để data không bị xé nhỏ và xé sai

Overcoming K-means Limitations

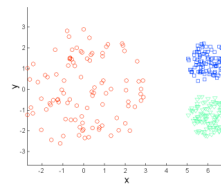


Original Points

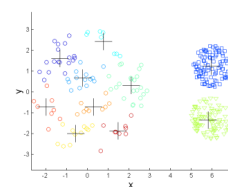


K-means Clusters

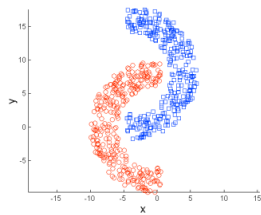
One solution is to use many clusters.
Find parts of clusters, but need to put together.



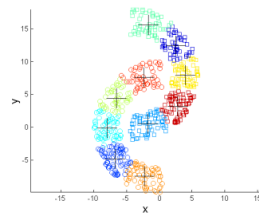
Original Points



K-means Clusters



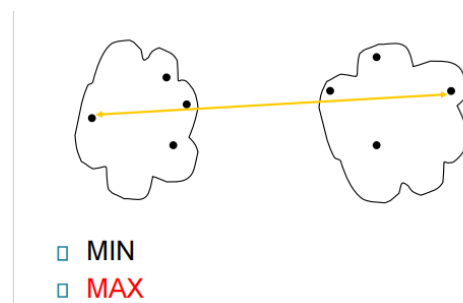
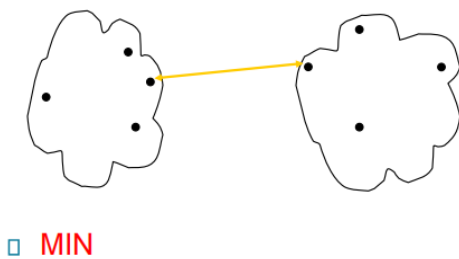
Original Points

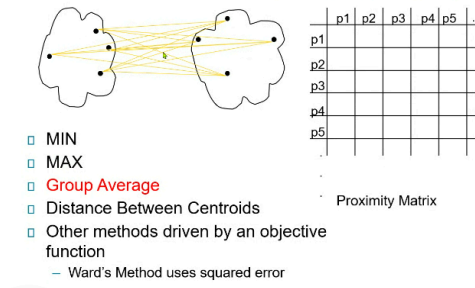


K-means Clusters

▼ C. Hierarchical Clustering:

- Cách tính similarity (tương đồng của 2 cụm):
 - Tính khoảng cách centroid (bởi vì group nó lớn quá $\rightarrow$ nên lấy cái centroid để dễ tính)





Min = single link

Max = complete link

▼ D. DBSCAN Algorithm:

Ví dụ cách hoạt động DBSCAN:

- Bản chất tính khoảng cách Euclid xem nó nhỏ hơn eps không, và cứ expand đi từng các điểm như thế để tạo thành các cụm).

Thuật toán DBSCAN(Dataset, eps, MinPts)

Input: Dataset (các điểm), eps (bán kính của Scan), MinPts (số điểm tối thiểu để gom thành 1 cụm)

Output: Nhãn cụm cho mỗi điểm (Cụm ID hoặc Noise)

cluster_id = 0

for each point p in Dataset:

p.label = UNVISITED # mặc định các point trong cụm data đều chưa được thăm

Đi kiểm tra từng điểm trong dataset

for each point p in Dataset:

if p.label != UNVISITED: continue # nếu nhãn "đã thăm" - continue

neighbors = find_neighbors(p, eps, Dataset) #bỏ vào hàm này - để tìm các điểm l

ân cận

if size(neighbors) < MinPts: # số point của điểm vừa tìm - có ít neighbor, tức là không đủ tạo cụm → noise

p.label = NOISE

continue

cluster_id = cluster_id + 1

p.label = cluster_id

#Tiếp tục mở rộng cụm

expand_cluster(p, neighbors, cluster_id, eps, MinPts, Dataset)

return cluster_labels

Mở rộng các cụm lại với nhau

Function expand_cluster(p, neighbors, cluster_id, eps, MinPts, Dataset)

for each point q in neighbors:

if q.label == UNVISITED: # nếu điểm lân cận - chưa thăm thì vào đây tính khoảng cách điểm so với cụm

q.label = cluster_id

new_neighbors = find_neighbors(q, eps, Dataset)

if size(new_neighbors) >= MinPts:

neighbors = neighbors ∪ new_neighbors #nếu điểm lân cận trong expand_cluster xuất hiện → gom vào 1 cụm

else if q.label == NOISE:

q.label = cluster_id #nếu không đủ điểm có thể là noise

Hàm để tìm các neighbor bên trong eps → phục vụ mục đích xác định được "Core Point"

Function find_neighbors(p, eps, Dataset)

neighbors = {} # mảng để lưu trữ các điểm lân cận, mục đích để điểm nào đã thăm rồi thì lưu vào mảng

for each point q in Dataset:

if distance(p, q) <= eps: #xét xem điểm đó có nằm trong khóa epsilon cho phép tạo cụm không

neighbors.add(q)

return neighbors

Example Walkthrough (Simplified Dataset)

Let's use a tiny 2D dataset to illustrate DBSCAN:

- Dataset X : $[(1, 1), (1.5, 1), (2, 1), (5, 5), (5.5, 5), (10, 10)]$, so $n = 6, d = 2$.
- Parameters: $eps = 1, minPts = 3$.

Step 1: Initialize

- All points are unvisited, labels = $[-1, -1, -1, -1, -1, -1]$, $C = 0$.

Step 2: Process First Point (1, 1)

- Mark (1, 1) as visited.
- Neighbors within $eps = 1$: (1.5, 1), (2, 1) (distance ≤ 1).
- Number of neighbors = 3 (including (1, 1)), which is $\geq minPts$.
- (1, 1) is a core point. Start a new cluster: $C = 1$.
- Assign (1, 1) to cluster 1 and expand the cluster using its neighbors (1.5, 1), (2, 1).

Step 3: Expand Cluster 1

- **Point (1.5, 1):**
 - Mark as visited.
 - Neighbors: (1, 1), (2, 1) (distance ≤ 1).
 - Number of neighbors = 3, so (1.5, 1) is a core point.
 - Assign (1.5, 1) to cluster 1.
 - Neighbors (1, 1), (2, 1) are already in the queue.
- **Point (2, 1):**
 - Mark as visited.
 - Neighbors: (1, 1), (1.5, 1).
 - Number of neighbors = 3, so (2, 1) is a core point.
 - Assign (2, 1) to cluster 1.
- Cluster 1 is now (1, 1), (1.5, 1), (2, 1). Labels: $[1, 1, 1, -1, -1, -1]$.

Step 4: Process Next Unvisited Point (5, 5)

- Mark (5, 5) as visited.
- Neighbors: (5.5, 5) (distance ≤ 1).
- Number of neighbors = 2 (including (5, 5)), which is $< minPts$.
- Label (5, 5) as noise (-1). Labels: [1, 1, 1, -1, -1, -1].

Step 5: Process Next Unvisited Point (5.5, 5)

- Mark (5.5, 5) as visited.
- Neighbors: (5, 5).
- Number of neighbors = 2, which is $< minPts$.
- Label (5.5, 5) as noise (-1). Labels: [1, 1, 1, -1, -1, -1].

Step 6: Process Next Unvisited Point (10, 10)

- Mark (10, 10) as visited.
- Neighbors: None within $eps = 1$.
- Number of neighbors = 1, which is $< minPts$.
- Label (10, 10) as noise (-1). Labels: [1, 1, 1, -1, -1, -1].

Step 7: Output

- Final labels: [1, 1, 1, -1, -1, -1].
- Cluster 1: (1, 1), (1.5, 1), (2, 1).
- Noise points: (5, 5), (5.5, 5), (10, 10).



DBSCAN cơ bản:

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) là một thuật toán phân cụm dựa trên mật độ. Nó nhóm các điểm dữ liệu nằm gần nhau (mật độ cao) thành cụm và xác định các điểm nằm ở vùng thưa thớt là nhiễu (outlier).

- **Cách hoạt động:**

- Xác định các **điểm lõi** (core points): Điểm có ít nhất min_pts điểm khác trong bán kính eps.
- Mở rộng cụm từ các điểm lõi bằng cách kết nối các điểm láng giềng.
- Điểm không thuộc cụm nào được đánh dấu là **nhiều**.

Thông số eps và min_pts:

1. eps (epsilon) - đơn vị được tính bằng Euclidean,

- Bán kính xác định vùng láng giềng của một điểm.
- Nếu eps quá nhỏ, nhiều điểm trở thành nhiễu; nếu quá lớn, các cụm có thể gộp lại thành một.

2. min_pts:

- Số điểm tối thiểu cần có trong bán kính eps để một điểm được coi là điểm lõi - **điểm lõi (core point)** và hình thành một cụm.
- Nếu min_pts quá thấp, nhiễu có thể bị gộp vào cụm; nếu quá cao, nhiều điểm bị coi là nhiễu.

Tóm lại: eps kiểm soát kích thước vùng mật độ, min_pts xác định mức độ "đồng đúc" cần thiết để hình thành cụm. Cả hai cần được chọn dựa trên phân bố dữ liệu để phân cụm hiệu quả

Quy tắc xác định nhiễu trong DBSCAN:

- Một điểm được gọi là **điểm lõi (core point)** nếu trong bán kính eps xung quanh nó có ít nhất min_pts điểm (bao gồm chính nó).
 - Ví dụ eps = 0,1 + min = 5 → nếu trong eps đủ 5 điểm thì sẽ thành 1 cluster.
- Một điểm được gọi là **điểm biên (border point)** nếu nó không phải điểm lõi nhưng nằm trong bán kính eps của một điểm lõi.

- Một điểm được gọi là **nhieu (noise)** nếu nó không phải điểm lõi và cũng không nằm trong bán kính eps của bất kỳ điểm lõi nào.



DBSCAN Mechanism:

- DBSCAN $O(n^2)$ → cơ chế kiểm tra "corepoint" nếu có rồi thì bỏ qua.
- nếu có bất kỳ điểm nào (core point hoặc border point) nằm trong vùng lân cận của một core point của cụm kia, DBSCAN sẽ hợp nhất hai cụm thành **một cụm lớn** chứa tất cả các điểm density-connected.

⇒ hình dung 2 đường tròn giao nhau (có thể giao nhau "Core" hoặc giao về "border point" cũng được tính là một cụm).

- **Neighbors** : các điểm lân cận - nằm trong tầm của bán kính eps
 - **Core Point**: Một điểm có ít nhất **MinPts** (ở đây là 5) điểm trong bán kính **eps**, bao gồm chính nó.
 - **Density-reachable**: Một điểm B là density-reachable từ điểm A nếu có một chuỗi các **core points** kết nối từ A đến B.
 - **Density-connected**: Hai điểm thuộc cùng một cụm nếu chúng đều density-reachable từ một core point chung.
-
- **Noise**: Các điểm nhieu thường là những điểm không thuộc cụm nào (do không có đủ min_pts láng giềng trong bán kính eps),



Để tính core point và cluster:

- Dùng Euclidean, Manhattan, Cosine similarity để tính toán .

Nếu **eps** quá nhỏ, nhiều điểm sẽ bị coi là noise vì không đủ điểm lân cận. Nếu **eps** quá lớn, các cụm có thể hợp nhất thành một cụm lớn.

- Nếu **eps** quá nhỏ, nhiều điểm sẽ bị coi là noise vì không đủ điểm lân cận.
- Nếu **eps** quá lớn, các cụm có thể hợp nhất thành một cụm lớn.

MinPts nhỏ (ví dụ, 2 hoặc 3) có thể dẫn đến các cụm nhỏ và nhiều noise. **MinPts** lớn yêu cầu mật độ cao hơn, dẫn đến ít cụm hơn nhưng ổn định hơn.

- **MinPts** nhỏ (ví dụ, 2 hoặc 3) có thể dẫn đến các cụm nhỏ và nhiều noise.
- **MinPts** lớn yêu cầu mật độ cao hơn, dẫn đến ít cụm hơn nhưng ổn định hơn.

Trong DBSCAN, sau khi chạy thuật toán, các nhãn (labels) của các điểm dữ liệu sẽ được tự động tạo ra. Cụ thể:

- Các điểm thuộc cụm sẽ được gán nhãn là các số không âm: 0, 1, 2, ... (mỗi số đại diện cho một cụm riêng biệt).
- Các điểm nhiễu (noise/outliers) sẽ được gán nhãn -1.

▼ E. Clustering Evaluation:

▼ So sánh khi nào dùng K-Means và DBSCAN

- K-means dùng khi dataset lớn + xác định được k clusters. DBSCAN dataset nhỏ + chưa biết cần bao nhiêu cluster.
- **K-means** assumes clusters are spherical (hình cầu) and requires specifying the number of clusters beforehand.
- **DBSCAN** can discover clusters of arbitrary shapes (các hình dạng random) and **automatically** determines the number of clusters based on data density (tự xác định số cụm dựa vào mật độ).

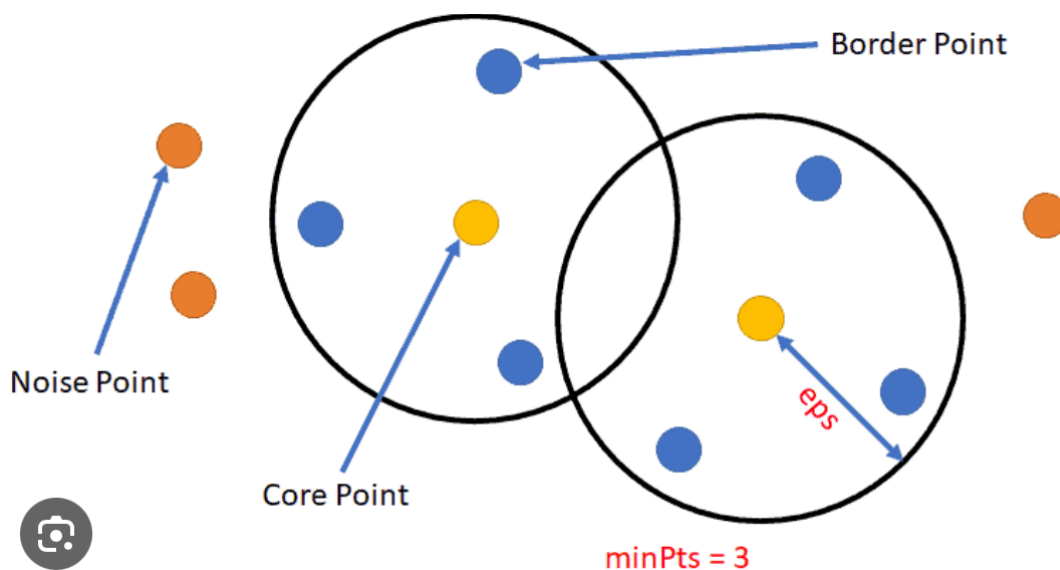
⇒ DBSCAN also excels at outlier detection, making it suitable for noisy datasets.

K-means:

- **Centroid-based:** It uses centroids (the mean of data points in a cluster) to define clusters.
- **Spherical clusters:** Assumes clusters are roughly spherical and of similar size.
- **Requires number of clusters:** You need to specify the number of clusters (k) in advance.
- **Sensitive to outliers:** Outliers can significantly affect cluster centroids.
- **Efficient for large datasets:** Generally faster than DBSCAN on large datasets.

DBSCAN:

- **Density-based:** Groups data points based on their density in the feature space.
- **Arbitrary shaped clusters:** Can discover clusters of irregular shapes and sizes.
- **Automatically determines number of clusters:** No need to specify the number of clusters in advance.
- **Handles noise well:** Outliers are identified as noise and are not assigned to any cluster.
- **Less efficient for large datasets:** Can be slower and less scalable than K-means for very large datasets.



▼ **Lecture 6 - Frequent Itemsets and Association Rules (mẫu phổ biến & kết hợp):**

▼ A. Frequent Itemsets:

▼ B. **Association Rules:**

▼ C. Result Post Processing:

▼ D. Alternative Items Set Computation:

▼ **Lecture 7 - (more infor, relate to Theory and Graph):**

▼ A. **Link Analysis Ranking:**

▼ B. **PageRank - Random walks:**

▼ C. **HITS Algo:**

▼ D. **Absorbing Random Walks and Label Propagation:**